Operating Systems – Assignment 2 (CO2)

Solutions to all questions (Q1–Q6)

Name: K. Vimala

Register number: 1924260099

GitHub repository:
https://github.com/vimalakkvimala436-design/operating-system-CSA0404

Environment used for all programs: Ubuntu 24.04.5 LTS, gcc 13.3.0, 2 CPUs, compiled with `gcc -std=c11 -Wall -Wextra -pthread` (with -O1/-O2 for benchmarks).

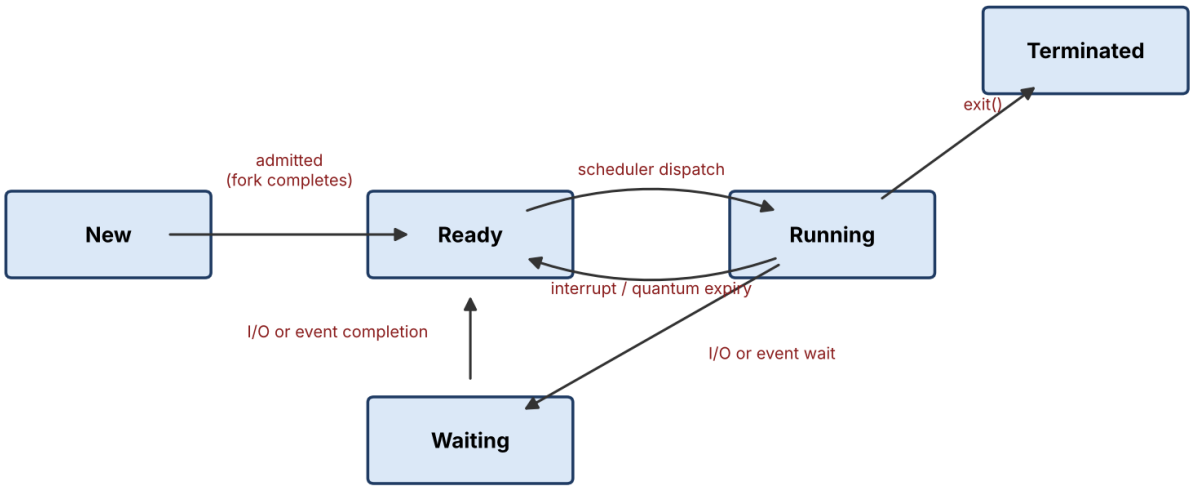
Q	Topic	Marks	Page
1	Processes and Process States – Online Examination Server	6	2
2	CPU Scheduling – Cloud Video-Encoding Service	8	6
3	Threads – Traffic-Camera Number-Plate Reader	6	14
4	Inter-Process Communication – Weather Station Data Pipeline	6	18
5	Synchronisation – Railway Ticket Booking	11	22
6	Deadlock Handling and the Banker's Algorithm – Laboratory Equipment Scheduler	13	27
	Total	50	

Operating Systems Assignment 2 – Task 1 (Q1)

Processes and Process States – Online Examination Server (CO2, K5 – Evaluate, 6 marks)

Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0, 2 CPUs, 8 GB RAM. Compile command: `gcc -std=c11 -Wall -Wextra -o fork_demo fork_demo.c` (benchmarks: `gcc -O1 -pthread`).

1. Process-state diagram



Transitions: New→Ready when the OS has created the PCB and memory image (end of fork); Ready→Running when the dispatcher picks the process; Running→Ready on timer interrupt or pre-emption; Running→Waiting when the session blocks on disk/network/timer; Waiting→Ready when the event completes; Running→Terminated on exit(). In the exam server, a student session is mostly in Waiting (waiting for keystrokes, auto-save disk writes, network), so it is rarely Running.

2. Process Control Block (PCB)

PCB field	Content	Linux equivalent (task_struct / /proc)
Process ID, parent ID	PID, PPID, user/group IDs	Pid, PPid, Uid in /proc/<pid>/status
Process state	new / ready / running / waiting / terminated	State: R, S, D, Z in /proc/<pid>/status
Program counter & CPU registers	Saved when the process is switched out	saved in kernel stack / thread_struct
CPU-scheduling info	Priority, queue pointers, time slice	nice, policy, /proc/<pid>/sched
Memory-management info	Page-table base, segment limits	mm_struct; VmSize, VmRSS, /proc/<pid>/maps
Accounting info	CPU time used, start time	utime, stime in /proc/<pid>/stat
I/O status info	Open-file table, pending I/O devices	/proc/<pid>/fd, wchan

3. Program: creating children with fork()

Each child models one student session: it prints its PID/PPID, sleeps (Waiting state) and exits; the parent reaps it with wait() so no zombie is left.

```
#define _GNU_SOURCE
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;unistd.h&gt;
#include &lt;sys/wait.h&gt;
#include &lt;string.h&gt;
int main(int argc, char *argv[]) {
```

```
    int n = (argc &gt; 1) ? atoi(argv[1]) : 5;
    int secs = (argc &gt; 2) ? atoi(argv[2]) : 3;
    printf("Parent PID=%d creating %d children (each lives %d s)\n", getpid(), n, secs);
    fflush(stdout);
    for (int i = 0; i &lt; n; i++) {
        pid_t pid = fork();
        if (pid &lt; 0) { perror("fork"); exit(1); }
        if (pid == 0) {                       /* child: one "student session" */
            printf("  child %d: PID=%d PPID=%d\n", i, getpid(), getppid());
            fflush(stdout);
            sleep(secs);                      /* WAITING state (blocked on timer/I-O) */
            exit(i);                          /* TERMINATED -&gt; zombie until parent waits */
        }
    }
    fflush(stdout);
    int status, done = 0;
    while (done &lt; n) {
        pid_t p = wait(&amp;status);
        if (p &gt; 0) { printf("parent: reaped PID=%d exit=%d\n", p, WEXITSTATUS(status));
done++; }
    }
    return 0;
}
```

Output of `./fork_demo 3 1`

```
Parent PID=276 creating 3 children (each lives 1 s)
  child 0: PID=277 PPID=276
  child 1: PID=278 PPID=276
  child 2: PID=279 PPID=276
parent: reaped PID=277 exit=0
parent: reaped PID=279 exit=2
parent: reaped PID=278 exit=1
```

Observation with ps and /proc (while 5 children are alive)

```
  PID  PPID STAT    VSZ   RSS WCHAN          CMD
  257   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
  258   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
  259   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
  260   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
  261   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
```

```
Name:■fork_demo
State:■S (sleeping)
Pid:■261
PPid:■255
VmSize:■    2712 kB
VmRSS:■     928 kB
voluntary_ctxt_switches:■1
nonvoluntary_ctxt_switches:■0
```

All five children show STAT **S** (interruptible sleep = Waiting state), PPID 255 (the parent), and WCHAN `hrtimer_nanosleep` (the kernel function they are blocked in). Each child shows ~928 kB RSS and 1 voluntary context switch (it gave up the CPU by sleeping) and 0 involuntary switches. Just after exit and before the parent's wait(), a child would show STAT **Z** (zombie = Terminated, PCB retained).

4. Evaluation: where does the time go with 2,000 active processes?

Measured on this machine (`bench.c`, `spawn2000.c`):

```
fork+exit+wait   : 245.7 us each
pthread create+join: 60.5 us each
process ctx switch (pipe ping-pong, 1 CPU): 1.26 us
thread  ctx switch (pipe ping-pong, 1 CPU): 1.38 us
```

Cost	Measurement / estimate	Evaluation
Process creation	fork+exit+wait ≈ 246 µs each → 2,000 logins ≈ 0.5 s of pure creation work	Login burst is serialised in the kernel (page-table copy, PCB setup). Noticeable but not the main problem.

Cost	Measurement / estimate	Evaluation
Context switch	≈ 1.3 µs direct cost (pipe ping-pong, one CPU). Indirect cost (TLB flush, cache misses) is larger and grows with working-set size.	With only 2 cores, 2,000 ready processes means each gets a time slice only every ~1,000 slices; round-trip scheduling latency is long and sessions appear to “hang”.
Memory per process	Minimal test process: ≈ 0.3 MB RSS; 2,000 running together confirmed (pgrep = 2001). A real exam session (runtime, libraries, buffers) is typically 20–50 MB private memory → 40–100 GB, beyond the 8 GB RAM here.	Memory is the real bottleneck: once RAM is exhausted the system swaps (thrashing) and everything stalls.
I/O wait	Sessions are I/O-bound; most are in Waiting. Auto-save and proctoring video all hit the same disk/network.	Waiting costs no CPU, but contention on shared disk/network queues lengthens each wait; a burst of 2,000 simultaneous saves serialises.

Judgement: A separate process per student is *not* a sound design at this scale. Isolation is good, but the per-process address space, kernel structures and switching overhead are paid 2,000 times for work that is mostly waiting. Sessions share nearly everything (code, exam questions, configuration), so duplicating the address space wastes memory and makes the CPU spend its time switching rather than serving.

5. Design decision

Option	Strengths	Weaknesses
One process per student	Strong isolation; a crash affects one student	Highest memory and creation cost; heavy switches; fails at 2,000
One thread per student	Cheap creation (≈ 60 µs vs 246 µs measured), few-KB stacks, shared data	One bug or crash can kill every session; needs locking for shared data; 2,000 threads still add scheduling overhead
Fixed pool of worker processes (event-driven)	Bounded memory and switching; isolation between workers; scales predictably	More complex design; queueing delay when all workers are busy

Decision: a fixed pool of worker processes, each running many lightweight threads (or an event loop) for sessions, plus a separate small process for proctoring video. Pool size ≈ number of cores × 2–4 (for example 8 workers) keeps context switching and memory bounded; threads/events inside each worker handle the thousands of mostly-waiting sessions cheaply; sessions are distributed across workers so that a crash loses only 1/N of students, and the supervisor restarts the worker while answers are auto-saved to the database. Video proctoring, which is CPU-heavy, is isolated in its own processes so it cannot starve answer-saving.

6. Alternative concept: one thread per session inside one server process

Solution sketch:

```
main():  listen(); loop { c = accept(); pthread_create(&t, &attr(stack=64KB), session, c); pthread_detach(t); }
session(c): loop { read request; update shared exam table under mutex; write response }
```

Aspect	Process per session	Thread per session
Creation time (measured)	≈ 246 µs (fork+exit+wait)	≈ 60 µs (pthread_create+join) → ~4× faster
Context switch (measured)	≈ 1.26 µs direct; plus TLB flush & cache loss on address-space change	≈ 1.38 µs in this virtualised sandbox (equal within noise); on real hardware thread switches avoid the page-table switch/TLB flush, so are generally cheaper
Memory per session	Own page tables + PCB + private data; about 0.3 MB minimum, tens of MB realistic	Own stack (a few KB to 64 KB committed) + small kernel structure; code, heap and exam data shared → 2,000 sessions ≈ tens to hundreds of MB
Communication	IPC (pipes, shared memory, sockets)	Direct shared variables, protected by mutex

Aspect	Process per session	Thread per session
Effect of a crash	Only that student's session dies	A segfault or corrupted heap kills the whole process: all 2,000 sessions lost
Time complexity	Creation O(size of address space) due to page-table copy; switch O(1) + TLB cost	Creation O(1) (no address-space copy); switch O(1)
Space complexity	O(n × M) where M = per-process memory	O(M + n × s) where s = stack size (s ■ M)

Conclusion: threads cut memory and creation cost sharply and are better than 2,000 processes, but they give up fault isolation. The pooled hybrid in Section 5 keeps most of the thread advantage while limiting the blast radius of a crash to one worker.

Note on measurements

Benchmarks ran in a virtualised cloud container with 2 CPUs, so absolute values will differ on other machines; the relative ordering (thread creation much cheaper than fork; memory dominating at 2,000 processes) is the result to note. The summed RSS of the 2,000 test processes counts shared library pages repeatedly, so it overstates true memory use; the 0.3 MB figure is the per-process RSS of a minimal program only.

Operating Systems Assignment 2 – Task 2 (Q2)

CPU Scheduling – Cloud Video-Encoding Service (CO2, K5 – Evaluate, 8 marks). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0. Structure follows the assignment's expected flow: *Model* → *Design* → *Implement* → *Measure* → *Compare* → *Evaluate* → *Recommend*.

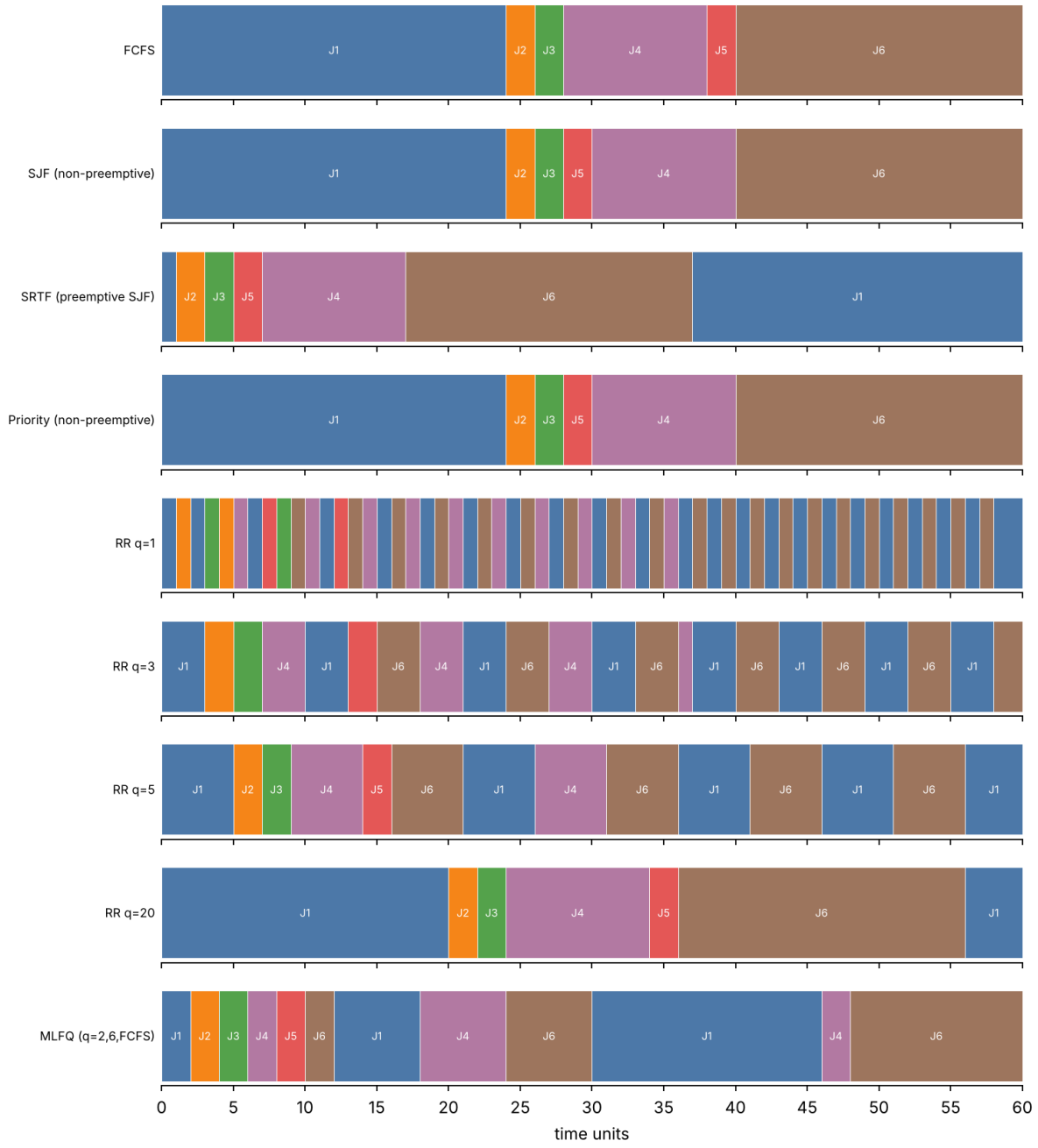
1. Model: the workload

The assignment does not supply a job table, so I model the service with six jobs (time unit = 1 s of encoding, scaled down; thumbnails 2 units, films 20+ units). Priority: lower number = higher priority. Total demand = 60 units, arrivals keep the CPU busy from time 0, so utilisation is 100% for every algorithm and the algorithms differ only in *who waits*.

Job	Type	Arrival	Burst	Priority
J1	Full film	0	24	3
J2	Thumbnail	1	2	1
J3	Thumbnail	2	2	1
J4	Medium clip	3	10	2
J5	Thumbnail	4	2	1
J6	Full film	5	20	3

Definitions: Turnaround (TAT) = completion – arrival; Waiting (WT) = TAT – burst; Response (RT) = first CPU time – arrival. A context switch is counted whenever the CPU passes to a different job. Priority scheduling is non-preemptive; Round Robin admits new arrivals before re-queueing the pre-empted job; MLFQ is defined in Section 6.

2. Design and results: Gantt charts, WT, TAT, RT



Gantt charts for all algorithms (computed by `sim.py`; RR also by the C program). Exact intervals and per-job values follow.

FCFS

`Gantt: |J1 0-24 |J2 24-26 |J3 26-28 |J4 28-38 |J5 38-40 |J6 40-60|`

Job	J1	J2	J3	J4	J5	J6	Average
WT	0	23	24	25	34	35	23.50
TAT	24	25	26	35	36	55	33.50
RT	0	23	24	25	34	35	23.50

Context switches: 5

SJF (non-preemptive)

`Gantt: |J1 0-24 |J2 24-26 |J3 26-28 |J5 28-30 |J4 30-40 |J6 40-60|`

Job	J1	J2	J3	J4	J5	J6	Average
WT	0	23	24	27	24	35	22.17

Job	J1	J2	J3	J4	J5	J6	Average
TAT	24	25	26	37	26	55	32.17
RT	0	23	24	27	24	35	22.17

Context switches: 5

SRTF (preemptive SJF)

```
Gantt: |J1 0-1 |J2 1-3 |J3 3-5 |J5 5-7 |J4 7-17 |J6 17-37 |J1 37-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	36	0	1	4	1	12	9.00
TAT	60	2	3	14	3	32	19.00
RT	0	0	1	4	1	12	3.00

Context switches: 6

Priority (non-preemptive)

```
Gantt: |J1 0-24 |J2 24-26 |J3 26-28 |J5 28-30 |J4 30-40 |J6 40-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	0	23	24	27	24	35	22.17
TAT	24	25	26	37	26	55	32.17
RT	0	23	24	27	24	35	22.17

Context switches: 5

RR q=1

```
Gantt: |J1 0-1 |J2 1-2 |J1 2-3 |J3 3-4 |J2 4-5 |J4 5-6 |J1 6-7 |J5 7-8 |J3 8-9 |J6 9-10 |J4 10-11 |J1 11-12 |J5 12-13
|J6 13-14 |J4 14-15 |J1 15-16 |J6 16-17 |J4 17-18 |J1 18-19 |J6 19-20 |J4 20-21 |J1 21-22 |J6 22-23 |J4 23-24 |J1 24-25
|J6 25-26 |J4 26-27 |J1 27-28 |J6 28-29 |J4 29-30 |J1 30-31 |J6 31-32 |J4 32-33 |J1 33-34 |J6 34-35 |J4 35-36 |J1 36-37
|J6 37-38 |J1 38-39 |J6 39-40 |J1 40-41 |J6 41-42 |J1 42-43 |J6 43-44 |J1 44-45 |J6 45-46 |J1 46-47 |J6 47-48 |J1 48-49
|J6 49-50 |J1 50-51 |J6 51-52 |J1 52-53 |J6 53-54 |J1 54-55 |J6 55-56 |J1 56-57 |J6 57-58 |J1 58-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	36	2	5	23	7	33	17.67
TAT	60	4	7	33	9	53	27.67
RT	0	0	1	2	3	4	1.67

Context switches: 58

RR q=3

```
Gantt: |J1 0-3 |J2 3-5 |J3 5-7 |J4 7-10 |J1 10-13 |J5 13-15 |J6 15-18 |J4 18-21 |J1 21-24 |J6 24-27 |J4 27-30 |J1 30-33
|J6 33-36 |J4 36-37 |J1 37-40 |J6 40-43 |J1 43-46 |J6 46-49 |J1 49-52 |J6 52-55 |J1 55-58 |J6 58-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	34	2	3	24	9	35	17.83
TAT	58	4	5	34	11	55	27.83
RT	0	2	3	4	9	10	4.67

Context switches: 21

RR q=5

```
Gantt: |J1 0-5 |J2 5-7 |J3 7-9 |J4 9-14 |J5 14-16 |J6 16-21 |J1 21-26 |J4 26-31 |J6 31-36 |J1 36-41 |J6 41-46 |J1 46-51
|J6 51-56 |J1 56-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	36	4	5	18	10	31	17.33
TAT	60	6	7	28	12	51	27.33
RT	0	4	5	6	10	11	6.00

Context switches: 13

RR q=20

Gantt: |J1 0-20 |J2 20-22 |J3 22-24 |J4 24-34 |J5 34-36 |J6 36-56 |J1 56-60|

Job	J1	J2	J3	J4	J5	J6	Average
WT	36	19	20	21	30	31	26.17
TAT	60	21	22	31	32	51	36.17
RT	0	19	20	21	30	31	20.17

Context switches: 6

MLFQ (q=2,6,FCFS)

Gantt: |J1 0-2 |J2 2-4 |J3 4-6 |J4 6-8 |J5 8-10 |J6 10-12 |J1 12-18 |J4 18-24 |J6 24-30 |J1 30-46 |J4 46-48 |J6 48-60|

Job	J1	J2	J3	J4	J5	J6	Average
WT	22	1	2	35	4	35	16.50
TAT	46	3	4	45	6	55	26.50
RT	0	1	2	3	4	5	2.50

Context switches: 11

3. Implement: Round Robin in C (Complete the Code, TODO 1)

TODO 1 was the dispatch step: take the head of the ready queue, run it for min(quantum, remaining), record first-run time, count context switches, add arrivals that came during the slice, then re-queue the job if unfinished. The rest of the program (job table, admission, statistics printing) is fixed. Because the assignment's skeleton was not included in the file I received, this is my own full program; if your faculty skeleton differs, paste the TODO block into it.

```
/* Task 2 - Round Robin scheduler (TODO 1 completed: the dispatch/quantum/requeue logic)
   Compile: gcc -std=c11 -Wall -Wextra -o rr rr.c      Run: ./rr &lt;quantum&gt; */
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#define N 6
typedef struct { const char *name; int at, bt, rem, first, ct; } Job;
int main(int argc, char *argv[]) {
    int q = (argc &gt; 1) ? atoi(argv[1]) : 3;
    if (q &lt;= 0) { fprintf(stderr, "quantum must be &gt; 0\n"); return 1; }
    Job j[N] = {{"J1-film",0,24,0,0,0},{"J2-thumb",1,2,0,0,0},{"J3-thumb",2,2,0,0,0},
                {"J4-medium",3,10,0,0,0},{"J5-thumb",4,2,0,0,0},{"J6-film",5,20,0,0,0}};
    int queue[1000], head = 0, tail = 0, t = 0, done = 0, next = 0, last = -1, cs = 0;
    for (int i = 0; i &lt; N; i++) { j[i].rem = j[i].bt; j[i].first = -1; }
    printf("Round Robin, quantum = %d\nGantt: ", q);
    while (done &lt; N) {
        while (next &lt; N &amp;&amp; j[next].at &lt;= t) queue[tail++] = next++;   /* admit arrivals */
        if (head == tail) { t++; continue; }                          /* CPU idle */
        /* TODO 1 (completed): dispatch head of queue, run for min(q, remaining) */
        int i = queue[head++];
        if (last != -1 &amp;&amp; last != i) cs++;                             /* context switch */
        last = i;
        if (j[i].first &lt; 0) j[i].first = t;                           /* response time */
        int run = (j[i].rem &lt; q) ? j[i].rem : q;
        printf("|%s %d-%d ", j[i].name, t, t + run);
        t += run; j[i].rem -= run;
        while (next &lt; N &amp;&amp; j[next].at &lt;= t) queue[tail++] = next++;   /* arrivals first ... */
        if (j[i].rem &gt; 0) queue[tail++] = i;                           /* ... then preempted job */
        else { j[i].ct = t; done++; }
    }
    printf("|\n\n%-10s %3s %3s %5s %5s %5s\n","Job","AT","BT","WT","TAT","RT");
    double w = 0, ta = 0, r = 0;
    for (int i = 0; i &lt; N; i++) {
        int tat = j[i].ct - j[i].at, wt = tat - j[i].bt, rt = j[i].first - j[i].at;
        printf("%-10s %3d %3d %5d %5d %5d\n", j[i].name, j[i].at, j[i].bt, wt, tat, rt);
        w += wt; ta += tat; r += rt;
    }
    printf("\nAvg WT=%.2f  Avg TAT=%.2f  Avg RT=%.2f  Context switches=%d\n", w/N, ta/N, r/N, cs);
    return 0;
}
```

Execution: quantum = 1 (full output)

```
Round Robin, quantum = 1
Gantt: |J1-film 0-1 |J2-thumb 1-2 |J1-film 2-3 |J3-thumb 3-4 |J2-thumb 4-5 |J4-medium 5-6 |J1-film 6-7 |J5-
thumb 7-8 |J3-thumb 8-9 |J6-film 9-10 |J4-medium 10-11 |J1-film 11-12 |J5-thumb 12-13 |J6-film 13-14 |J4-
medium 14-15 |J1-film 15-16 |J6-film 16-17 |J4-medium 17-18 |J1-film 18-19 |J6-film 19-20 |J4-medium 20-21
|J1-film 21-22 |J6-film 22-23 |J4-medium 23-24 |J1-film 24-25 |J6-film 25-26 |J4-medium 26-27 |J1-film 27-28
|J6-film 28-29 |J4-medium 29-30 |J1-film 30-31 |J6-film 31-32 |J4-medium 32-33 |J1-film 33-34 |J6-film 34-35
|J4-medium 35-36 |J1-film 36-37 |J6-film 37-38 |J1-film 38-39 |J6-film 39-40 |J1-film 40-41 |J6-film 41-42
|J1-film 42-43 |J6-film 43-44 |J1-film 44-45 |J6-film 45-46 |J1-film 46-47 |J6-film 47-48 |J1-film 48-49 |J6-
film 49-50 |J1-film 50-51 |J6-film 51-52 |J1-film 52-53 |J6-film 53-54 |J1-film 54-55 |J6-film 55-56 |J1-film
56-57 |J6-film 57-58 |J1-film 58-59 |J1-film 59-60 |

Job         AT  BT    WT   TAT    RT
J1-film      0  24    36    60     0
J2-thumb     1   2     2     4     0
J3-thumb     2   2     5     7     1
J4-medium    3  10    23    33     2
J5-thumb     4   2     7     9     3
J6-film      5  20    33    53     4

Avg WT=17.67  Avg TAT=27.67  Avg RT=1.67  Context switches=58
```

Execution: quantum = 20 (full output)

```
Round Robin, quantum = 20
Gantt: |J1-film 0-20 |J2-thumb 20-22 |J3-thumb 22-24 |J4-medium 24-34 |J5-thumb 34-36 |J6-film 36-56 |J1-film
56-60 |

Job         AT  BT    WT   TAT    RT
```

```
J1-film      0  24    36    60     0
J2-thumb     1   2    19    21    19
J3-thumb     2   2    20    22    20
J4-medium    3  10    21    31    21
J5-thumb     4   2    30    32    30
J6-film      5  20    31    51    31

Avg WT=26.17  Avg TAT=36.17  Avg RT=20.17  Context switches=6
```

The C results match the Python simulator exactly for q = 1, 3, 5, 20 (e.g. q=1: Avg WT 17.67, 58 switches; q=20: Avg WT 26.17, 6 switches).

4. Measure and compare

Algorithm	Avg WT	Avg TAT	Avg RT	Ctx sw	Thumbnail avg WT (J2,J3,J5)	Film avg WT (J1,J6)
FCFS	23.50	33.50	23.50	5	27.00	17.50
SJF (non-preemptive)	22.17	32.17	22.17	5	23.67	17.50
SRTF (preemptive SJF)	9.00	19.00	3.00	6	0.67	24.00
Priority (non-preemptive)	22.17	32.17	22.17	5	23.67	17.50
RR q=1	17.67	27.67	1.67	58	4.67	34.50
RR q=3	17.83	27.83	4.67	21	4.67	34.50
RR q=5	17.33	27.33	6.00	13	6.33	33.50
RR q=20	26.17	36.17	20.17	6	23.00	33.50
MLFQ (q=2,6,FCFS)	16.50	26.50	2.50	11	2.33	28.50

Reading the table. FCFS, non-preemptive SJF and priority all make thumbnails wait behind J1 (the convoy effect): thumbnail WT is 24–27 units, i.e. 12–13× their own 2-unit runtime — exactly the customers' complaint. SRTF gives the best averages (WT 9.00) because short jobs always run first, but it pushes the film J1 to the very end (WT 36). RR and MLFQ keep thumbnails at 2–6 units WT.

5. Evaluate

5.1 Effect of the Round Robin quantum (measured)

Quantum	Avg WT	Avg TAT	Avg RT	Context switches	Behaviour
1	17.67	27.67	1.67	58	Best response, but 58 switches; with a real switch cost the overhead would be large
3	17.83	27.83	4.67	21	Good balance; thumbnails (2 units) finish in their first slice
5	17.33	27.33	6.00	13	Lowest WT here, fewer switches, slower first response
20	26.17	36.17	20.17	6	Quantum ≥ most bursts → degenerates towards FCFS; thumbnails wait behind 20-unit slices

Small quantum → good response but many switches (overhead ∝ burst/q); large quantum → few switches but FCFS-like convoy behaviour. A common rule is to pick q slightly larger than the length of a typical short job (here q = 3–5, so thumbnails complete in one slice).

5.2 Starvation in SJF/priority, and ageing

SJF and priority always pick the best-ranked job, so a stream of short or high-priority jobs can postpone a long or low-priority job indefinitely. Demonstration (`starve.py`): a low-priority job L (burst 4, priority 5) arrives at time 0, while high-priority 2-unit jobs (priority 1) arrive every 2 units for 40 units, saturating the CPU, with pre-emptive priority scheduling.

Policy	L first runs at	L finishes at
Priority, no ageing	t = 40 (only when the stream ends)	44

Policy	L first runs at	L finishes at
Priority + ageing (priority number –1 per 5 units waited)	t = 20	24

Ageing raises a job's priority the longer it waits, so its waiting time is bounded (here after 20 units L reaches priority 1 and, being the oldest, wins ties). In the main job set, SJF would starve J1/J6 if short jobs kept arriving; non-preemptive SJF here avoided it only because the arrivals were finite.

5.3 How a multilevel feedback queue treats short and long jobs

New jobs enter the top queue with a short quantum. A job that finishes within it (thumbnail) leaves immediately with tiny wait. A job that uses its whole quantum is demoted to a queue with a longer quantum, and finally to a FCFS queue, so long jobs run in bigger slices (few switches) but only when higher queues are empty. Short/interactive jobs are therefore favoured without knowing burst lengths in advance (unlike SJF/SRTF).

6. Alternative concept: MLFQ instead of a single Round Robin queue

Design: Q0 quantum 2, Q1 quantum 6, Q2 FCFS. New jobs go to Q0; using the full quantum demotes a job one level; a higher queue pre-empts a lower one. Pseudocode:

```
on arrival(job):      job.level = 0; enqueue(Q[0], job)
schedule():           for L in 0..2: if Q[L] not empty: return dequeue(Q[L])   # at most 3 checks -> O(1)
on quantum_end(job):  if job.level < 2: job.level += 1;  enqueue(Q[job.level], job)
on arrival to higher level while lower job runs: pre-empt, keep job at head of its own queue
optional ageing:      if job waits > T: job.level -= 1
```

Metric (same job set)	Single RR q=3	Single RR q=5	MLFQ (2,6,FCFS)
Avg WT / TAT / RT	17.83 / 27.83 / 4.67	17.33 / 27.33 / 6.00	16.50 / 26.50 / 2.50
Thumbnails avg WT	4.67	6.33	2.33
Films (J1, J6) avg WT	34.50	33.50	28.50
Context switches	21	13	11

In this workload MLFQ gave short jobs the lowest wait (2.33), improved the films' wait too (28.5 vs ≈34) because slices grow and fewer switches occur, and had fewer switches than RR q=3. Caveat: this is one small job set; MLFQ's advantage is largest when many short jobs keep arriving.

Cost	SJF	Single RR	MLFQ
Decision time	O(n) scan of ready jobs (O(log n) with a heap)	O(1) (queue head)	O(1) per decision: ≤ k queue checks, k = 3 constant
Space	O(n)	O(n)	O(n) + O(k) queue headers
Needs burst length in advance?	Yes (must be predicted)	No	No (learns from behaviour)
Starvation risk	High for long jobs	None	Possible for long jobs in bottom queue → fix with ageing / periodic boost

Own measurement of per-decision selection cost (`scal.py`, Python in this sandbox; values illustrate growth, not C speed):

```
n=   1000  SJF scan      33.56 us   MLFQ/RR pick  0.135 us
n=  10000  SJF scan     306.81 us   MLFQ/RR pick  0.121 us
n= 100000  SJF scan    3633.33 us   MLFQ/RR pick  0.182 us
```

The SJF scan grows linearly (≈ 34 µs → 307 µs → 3.6 ms for ×10 more jobs) while the queue-based pick stays flat (≈ 0.12–0.18 µs), confirming O(n) vs O(1).

7. Recommend: scheduling policy for the video service

Recommendation: a multilevel feedback queue (Q0 q≈2, Q1 q≈6, Q2 FCFS) with ageing/periodic priority boost, on every server. Evidence from the calculated averages:

Policy	Avg WT	Thumbnail WT	Film WT	Verdict
FCFS	23.50	27.00	17.50	Convoy effect – thumbnails wait 27 units: rejected
SJF / Priority (non-preemptive)	22.17	23.67	17.50	Still blocked behind running film; starvation risk

Policy	Avg WT	Thumbnail WT	Film WT	Verdict
SRTF	9.00	0.67	24.00	Best averages, but needs burst prediction, many arrivals can starve films, high decision cost O(n)
RR q=3 / q=5	17.83 / 17.33	4.67 / 6.33	34.50 / 33.50	Fair and simple, but films pay heavily; q must be tuned
MLFQ	**16.50**	**2.33**	**28.50**	**Short jobs nearly immediate, no burst prediction, O(1) decisions, 11 switches**

It meets both goals: small jobs no longer wait behind films (WT 2.33 vs 27 under FCFS), and the CPU stays 100% busy because there is always work in some queue. Its weak point is starvation of long films when thumbnails keep arriving; the ageing/boost rule bounds that wait (Section 5.2). SRTF has the best raw average but is rejected because job lengths are not known exactly in a real service and long films would be starved.

Limitations

Results come from one 6-job scenario I defined (the assignment gave none); averages will change with a different job set. Context-switch *cost* is counted, not timed, and the decision-cost timing used Python. Run the programs yourself and commit them (`Task02/rr.c, sim.py, starve.py, scal.py`, outputs) as the assignment requires.

Operating Systems Assignment 2 – Task 3 (Q3)

Threads – Traffic-Camera Number-Plate Reader (CO2, K5 – Evaluate, 6 marks). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0, Intel Xeon @ 2.1 GHz with **2 CPUs**. Compile: `gcc -std=c11 -O1 -Wall -Wextra -pthread`. Structure follows the expected flow *Model* → *Design* → *Implement* → *Measure* → *Compare* → *Evaluate* → *Recommend*.

1. Model

64 cameras each deliver a frame that must be processed (plate recognition). The first version handles cameras one after another, so total time = 64 × per-frame time and falls behind at peak. Frames are independent of each other, so the work is parallelisable; the only shared items are the work index and the result counter. I model recognition as a fixed CPU loop per frame (≈ 7.8 ms) and a second workload that adds a 20 ms wait for the frame to arrive over the network, to represent I/O.

2. Which resources threads share and which they own

Shared by all threads of a process	Private to each thread
Code (text) segment, global/static data, heap	Thread ID and its own stack (local variables, call frames)
Open files, sockets, file-descriptor table	CPU register set including the program counter and stack pointer
Signal handlers, working directory, process ID, user/group IDs	Thread-local storage (`__thread`), errno, signal mask, scheduling priority/state

Sharing is why thread creation and switching are cheap (no new address space) and why shared data such as the plate counter need locking.

3. User-level vs kernel-level threads, and the mapping models

	User-level threads	Kernel-level threads
Managed by	Thread library in user space; kernel sees one process	Operating-system kernel
Create/switch cost	Very low (no system call)	Higher (system call, kernel scheduling), still far cheaper than a process
Blocking I/O	One blocking call blocks all threads of the process	Only the calling thread blocks; others keep running
Multicore	Cannot run in parallel on several cores	Can run truly in parallel on several cores

Model	Mapping	Advantage	Drawback
Many-to-one	Many user threads → 1 kernel thread	Cheap management	No parallelism; one blocking call stops all
One-to-one	Each user thread → 1 kernel thread	True parallelism, blocking does not stop others	Each thread costs a kernel thread; limit on count
Many-to-many	M user threads → N ≤ M kernel threads	Parallelism with a bounded kernel-thread count	Complex to implement; rarely used now

Linux POSIX threads (NPTL) use one-to-one. For this problem that is the right model: camera threads blocked on the network must not block the others, and CPU-heavy recognition must be able to use every core.

4. Implement: multithreaded program

`plates.c` runs the 64 camera frames in three modes: sequential, **threads** (a pool of 4/8/16 workers pulling the next camera index under a mutex, or 64 = one thread per camera) and **processes** (alternative in Section 8, results combined through a pipe). Usage: `./plates thr 8 cpu`.

```
/* Task 3 - Number-plate reader: 64 cameras, POSIX threads vs processes.
   Compile: gcc -std=c11 -O1 -Wall -Wextra -pthread -o plates plates.c
   Usage:   ./plates &lt;mode&gt; &lt;workers&gt; &lt;workload&gt;
            mode: seq | thr | proc   workload: cpu | mixed   */
#define _GNU_SOURCE
```

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;unistd.h&gt;
#include &lt;pthread.h&gt;
#include &lt;sys/wait.h&gt;
#include &lt;time.h&gt;
#define CAMERAS 64
#define WORK 6000000UL          /* "image processing" loop per camera */
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&amp;t);return t.tv_sec+t.tv_nsec*1e-9;}
static int mixed = 0;
static volatile unsigned long sink;
static unsigned long process_frame(int cam){      /* CPU work (+ optional I/O wait: camera frame fetch) */
    if (mixed) usleep(20000);                      /* 20 ms waiting for the frame over the network */
    unsigned long h = 14695981039346656037UL ^ cam;
    for (unsigned long i = 0; i &lt; WORK; i++) h = (h ^ i) * 1099511628211UL;
    return h;
}
static int next_cam = 0;                           /* shared work counter */
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static long plates_ok = 0;                         /* shared result counter (protected) */
static void *worker(void *a){
    (void)a;
    for(;;){
        pthread_mutex_lock(&amp;m); int c = next_cam++; pthread_mutex_unlock(&amp;m);
        if (c &gt;= CAMERAS) break;
        unsigned long h = process_frame(c); sink = h;
        pthread_mutex_lock(&amp;m); plates_ok++; pthread_mutex_unlock(&amp;m);
    }
    return NULL;
}
static void *one_cam(void *a){ int c=(int)(long)a; sink=process_frame(c);
    pthread_mutex_lock(&amp;m); plates_ok++; pthread_mutex_unlock(&amp;m); return NULL; }
int main(int argc,char**argv){
    const char*mode = argc&gt;1?argv[1]:"seq"; int w = argc&gt;2?atoi(argv[2]):1;
    mixed = (argc&gt;3 &amp;&amp; !strcmp(argv[3],"mixed"));
    double t0=now();
    if(!strcmp(mode,"seq")){ for(int c=0;c&lt;CAMERAS;c++){sink=process_frame(c);plates_ok++;} }
    else if(!strcmp(mode,"thr")){
        pthread_t th[CAMERAS];
        if(w&gt;=CAMERAS){ for(long c=0;c&lt;CAMERAS;c++) pthread_create(&amp;th[c],NULL,one_cam,(void*)c);
                        for(int c=0;c&lt;CAMERAS;c++) pthread_join(th[c],NULL); }
        else { for(int i=0;i&lt;w;i++) pthread_create(&amp;th[i],NULL,worker,NULL);
               for(int i=0;i&lt;w;i++) pthread_join(th[i],NULL); }
    } else {                                       /* processes: results combined through a pipe */
        int fd[2]; pipe(fd);
        for(int i=0;i&lt;w;i++) if(fork()==0){
            long cnt=0; for(int c=i;c&lt;CAMERAS;c+=w){ sink=process_frame(c); cnt++; }
            write(fd[1],&amp;cnt,sizeof cnt); _exit(0); }
        for(int i=0;i&lt;w;i++){ long cnt; read(fd[0],&amp;cnt,sizeof cnt); plates_ok+=cnt; }
        while(wait(NULL)&gt;0);
    }
    printf("%-4s workers=%-2d %-5s frames=%ld time=%.3f s\n",mode,w,mixed?"mixed":"cpu",plates_ok,now()-t0);
    return 0;
}
```

Sample execution

```
$ ./plates seq 1 cpu
seq  workers=1  cpu   frames=64 time=0.501 s
$ ./plates thr 8 cpu
thr  workers=8  cpu   frames=64 time=0.253 s
$ ./plates thr 64 mixed
thr  workers=64 mixed frames=64 time=0.273 s
```

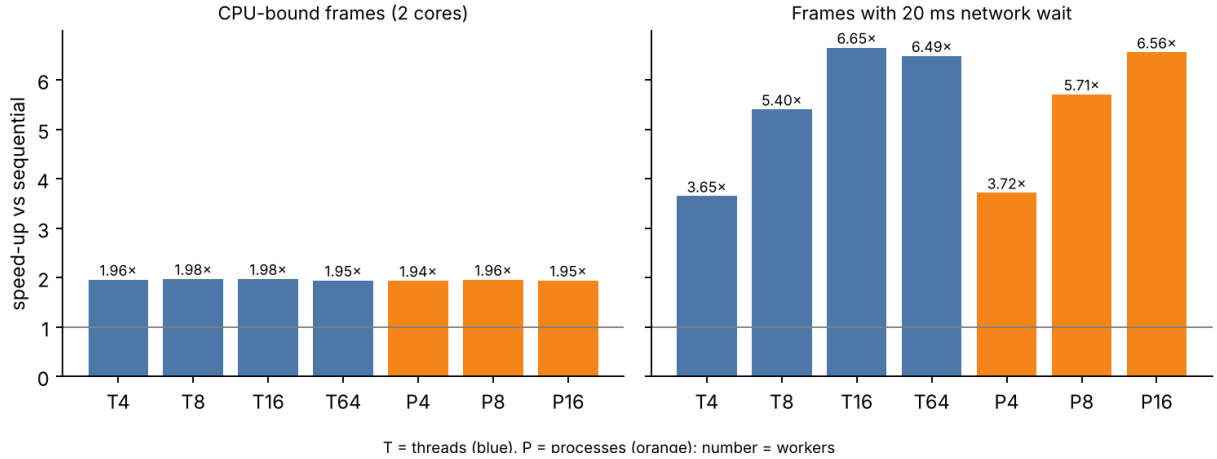
All 64 frames are always processed (frames=64), so the parallel runs produce the same result as the sequential one.

5. Measure: speed-up for each pool size

Each value is the **median of 5 runs** (`bench.sh`). Speed-up S = T_{seq} / $T_{parallel}$; efficiency = S ÷ number of cores.

Configuration	CPU-bound time (s)	Speed-up	Efficiency (÷2 cores)	With network wait: time (s)	Speed-up
Sequential (1 thread)	0.496	1.00×	50%	1.810	1.00×
Threads, pool of 4	0.253	1.96×	98%	0.496	3.65×

Configuration	CPU-bound time (s)	Speed-up	Efficiency (÷2 cores)	With network wait: time (s)	Speed-up
Threads, pool of 8	0.251	1.98×	99%	0.335	5.40×
Threads, pool of 16	0.251	1.98×	99%	0.272	6.65×
Threads, pool of 64 (one per camera)	0.255	1.95×	97%	0.279	6.49×
Processes, 4	0.256	1.94×	97%	0.487	3.72×
Processes, 8	0.253	1.96×	98%	0.317	5.71×
Processes, 16	0.255	1.95×	97%	0.276	6.56×



6. Evaluate

6.1 Why the speed-up stops increasing

CPU-bound workload: 4 threads give 1.96×, and 8, 16 and 64 threads give the same ≈ 1.95–1.98×. The machine has only **2 cores**, so at most 2 threads can run at the same instant; extra threads just wait for a core, and more threads add creation and switching cost without adding computing power. The ceiling is the number of cores (speed-up ≤ p cores), and by Amdahl's law $S \le 1/((1-f) + f/p)$, where the serial fraction (1–f) — creating threads, taking the work-index mutex, joining — is tiny here, so we reach ≈ 98% of the 2× limit.

Workload with network wait: while a thread waits for its frame, the core is free for another thread, so more threads than cores *do* help: 3.65× (4 threads) → 5.4× (8) → 6.65× (16). It then stops growing, and one thread per camera (64) is slightly slower (6.49×) than 16: the total CPU work (≈ 0.5 s over 2 cores = 0.25 s) is a floor that waiting cannot hide, and 64 threads add creation, stack memory and scheduling overhead. Rule of thumb: threads ≈ cores × (1 + wait time ÷ compute time) = 2 × (1 + 20/7.8) ≈ 7 for balance, and measurement shows gains continuing up to 16 as the ratio varies.

6.2 What happens to shared counters without protection

`race.c`: two threads each add 1 to a shared counter 1,000,000 times.

```
expected      : 2000000
unprotected   : 1951038  (lost 48962 updates)
mutex         : 2000000
atomic        : 2000000
```

Explanation: `counter++` is three steps — load, add, store. When two threads interleave (both load 5, both store 6) one update is lost. This is a **race condition**: the result depends on timing and changes from run to run (here 48,962 updates lost, ≈ 2.4%). A mutex (or an atomic increment) makes the update indivisible and gives the exact 2,000,000. The cost: the mutex serialises that tiny section, so I keep the critical section short (only the index fetch and counter increment, never the image processing).

6.3 Evaluation of the thread models for this system

With 1:1 kernel threads, both workloads gain from threads. Many-to-one would remain at ≈ 1× on a multicore machine for the CPU work (no parallelism) and a blocking frame read would freeze all cameras. Many-to-many

would give no extra benefit over a bounded pool at 1:1 for 64 cameras.

7. Design decision

Use a bounded pool of kernel (1:1) threads, sized by workload: about the number of cores for pure CPU recognition, and about 2–4× the cores (8–16 on this machine) when frames come over the network. Do not create one thread per camera as the default: 64 threads (6.49×) are no faster than 16 (6.65×) and cost more memory, and on a city system with thousands of cameras one-per-camera would not scale. Shared counters must be protected by a mutex or atomic operation.

8. Alternative: several processes instead of threads

Solution: `./plates proc N` forks N child processes; each processes cameras i, i+N, i+2N… and writes its frame count to a pipe; the parent reads N counts and adds them. No locking is needed because nothing is shared — results travel by IPC.

Aspect	Threads	Processes
Speed-up measured (CPU / with wait)	4: 1.96× / 3.65×; 8: 1.98× / 5.40×; 16: 1.98× / 6.65×	4: 1.94× / 3.72×; 8: 1.96× / 5.71×; 16: 1.95× / 6.56×
Creation time (measured in Task 1, same machine)	≈ 60 µs per pthread_create+join	≈ 246 µs per fork+exit+wait (≈ 4× slower)
Switching time	≈ 1.4 µs (same address space, no TLB flush on real hardware)	≈ 1.3 µs measured; plus TLB flush/cache loss with a different address space
Memory	Shared code, heap and data; each thread adds only a stack (a few KB committed)	Each process has its own page tables and private data (copy-on-write at fork); more memory per worker
Combining results	Shared variable + mutex/atomic (fast, but race risk)	IPC: pipe/shared memory/socket (system calls, copying; no race on data)
Fault isolation	A crash kills all threads	A crash kills only that process
Time complexity	Create O(1) per thread	Create O(address-space size) for page-table copy
Space complexity	O(M + p·s), s = stack	O(p·M)

Finding: for 64 frames of ~8 ms work, the measured end-to-end times of processes and threads are almost identical (e.g. 0.253 s vs 0.251 s CPU-bound with 8 workers), because creation cost (≈ 0.25 ms × a few workers) is tiny compared with 0.25 s of work. Processes would win when isolation matters (a bad frame crashing the recogniser must not stop other cameras) and threads win when creation is frequent, memory is tight or results must be shared at high rate. For long-running, small pools the two perform the same, so the choice should be driven by isolation vs sharing, not raw speed.

Limitations

Measured on a 2-core virtualised cloud container; on an 8- or 16-core machine the CPU-bound speed-up would keep growing up to the core count, then flatten. Frame processing is a synthetic loop, not real plate recognition. Run the programs on your own machine, commit them with outputs (`Task03/plates.c, race.c, bench.sh`) and use your own timings, with your name, register number and GitHub link added.

Operating Systems Assignment 2 – Task 4 (Q4)

Inter-Process Communication – Weather Station Data Pipeline (CO2, K5 – Evaluate, 6 marks). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0, 2 CPUs. Compile: `gcc -std=c11 -O2 -Wall -Wextra -pthread`. Structure follows the expected flow *Model* → *Design* → *Implement* → *Measure* → *Compare* → *Evaluate* → *Recommend*.

1. Model

A **sensor process** (producer) reads sensors and must pass a continuous stream of readings to an **analysis process** (consumer) without losing any. The two processes have separate address spaces, so they need an IPC mechanism. Two methods are implemented: (A) a pipe, (B) shared memory protected by semaphores.

2. Concepts to determine

Question	Answer
How does a pipe signal end of data?	When the writer closes **all** copies of the write end, the reader's `read()` returns **0** (EOF) after the remaining data is drained. The reader therefore loops until read() returns 0. If any process still holds the write end open, read() blocks instead.
Shared memory vs message passing	**Message passing** (pipe, message queue, socket): data is sent through the kernel with send/receive operations; the kernel handles synchronisation and buffering; simple and safe, slower for large data. **Shared memory**: both processes map the same memory region and read/write it directly; the kernel is involved only in setting it up, so it is fastest, but the programmer must provide synchronisation.
Which method copies the data?	**Pipe copies**: user buffer → kernel pipe buffer (copy 1), kernel buffer → reader's buffer (copy 2). **Shared memory does not copy through the kernel**: the consumer reads the same physical pages the producer wrote (the producer still writes the data into the region once).
Which needs extra synchronisation, and why?	**Shared memory**. Nothing stops the producer overwriting a slot the consumer has not read, or the consumer reading a slot not yet written; this is a race condition / bounded-buffer problem, solved here with two semaphores (`empty`, `full`). A pipe has the kernel's blocking behaviour built in: write() blocks when the pipe is full and read() blocks when it is empty.

3. Implement A: pipe – Complete the Code (TODO 1)

TODO 1 is closing the unused pipe ends: the child closes `fd[1]` (it only reads) and the parent closes `fd[0]` (it only writes). Then the parent closes `fd[1]` after sending the last reading so the child gets EOF.

```
/* Task 4 - Weather station: sensor process -&gt; analysis process through a pipe.
   TODO 1 (completed): close the unused pipe ends in each process.
   gcc -std=c11 -Wall -Wextra -o pipe_demo pipe_demo.c
   ./pipe_demo          normal run
   ./pipe_demo noclose  parent does NOT close fd[1]  -&gt; child never sees EOF */
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;unistd.h&gt;
#include &lt;sys/wait.h&gt;
typedef struct { int id; double temp, humidity; } Reading;
int main(int argc, char *argv[]) {
    int noclose = (argc &gt; 1 &amp;&amp; strcmp(argv[1], "noclose") == 0);
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid &lt; 0) { perror("fork"); return 1; }
    if (pid == 0) {                                  /* analysis process (reader) */
        close(fd[1]);                                /* TODO 1: child closes write end */
        Reading r; int n = 0; double sum = 0; ssize_t k;
        while ((k = read(fd[0], &amp;r, sizeof r)) == (ssize_t)sizeof r) { n++; sum += r.temp; }
        /* read() returns 0 (EOF) only when EVERY write end is closed */
        printf("analysis: read() returned %zd (EOF) after %d readings, mean temp %.2f\n", k, n, n ? sum / n : 0);
        close(fd[0]);
        return 0;
    }
    close(fd[0]);                                    /* TODO 1: parent closes read end */
    for (int i = 1; i &lt;= 10; i++) {                   /* sensor process (writer) */
        Reading r = {i, 20.0 + i * 0.5, 60.0 - i};
        write(fd[1], &amp;r, sizeof r);
    }
    if (!noclose) close(fd[1]);                      /* signals end of data to the reader */
```

```
    else printf("sensor: NOT closing write end, waiting for child...\n");
    fflush(stdout);
    wait(NULL);
    printf("sensor: child finished\n");
    return 0;
}
```

Execution – normal run

```
analysis: read() returned 0 (EOF) after 10 readings, mean temp 22.75
sensor: child finished
```

All 10 readings arrive and the analysis process sees read() = 0 (EOF) and terminates.

Expected-output experiment: remove close(fd[1]) in the parent

```
sensor: NOT closing write end, waiting for child...
exit status: 124 (124 = killed by timeout, child blocked in read())
```

Why the child never finishes: a pipe reports EOF only when *no* process has the write end open. The parent still holds `fd[1]` open (it is blocked in `wait()`, so it can never close it later), so the kernel cannot tell that no more data will come. The child's `read()` therefore blocks forever waiting for data, and the parent's `wait()` waits for the child: a **deadlock**. The run above had to be killed by `timeout` after 3 s (exit status 124). Similarly, if the child did not close its own copy of `fd[1]`, it would also keep the pipe open and never see EOF.

4. Implement B: shared memory with semaphores

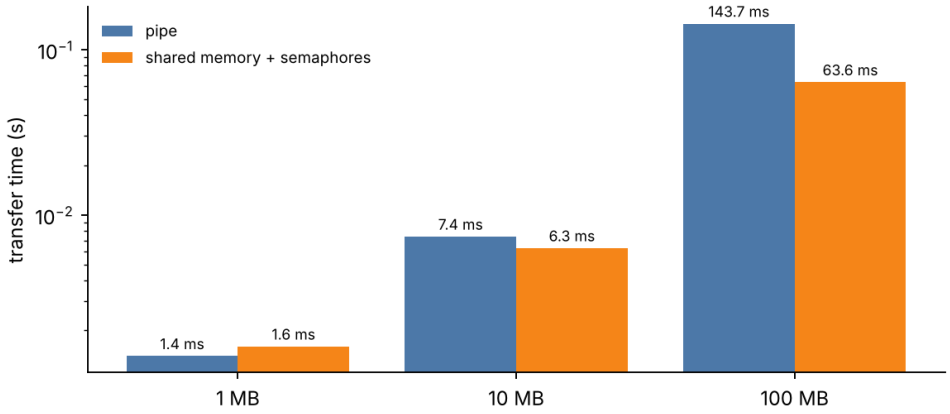
`ipc.c` sends N MB from a producer process to a consumer process using either a pipe (64 KB chunks) or a shared-memory ring buffer of 8 slots × 64 KB created with `mmap(MAP_SHARED|MAP_ANONYMOUS)`. Two process-shared POSIX semaphores synchronise the slots: `empty` counts free slots (initially 8), `full` counts filled slots (initially 0). A slot length of 0 marks end of data. The consumer calculates a checksum, so lost or corrupted data would be detected.

```
/* Transfer N MB from a producer process to a consumer process.
   pipe : kernel pipe, 64 KB chunks              (copy user-&gt;kernel-&gt;user)
   shm  : shared ring buffer + POSIX semaphores  (no kernel copy of the payload)
   gcc -std=c11 -O2 -Wall -Wextra -pthread -o ipc ipc.c      ./ipc &lt;pipe|shm&gt; &lt;MB&gt; */
#define _GNU_SOURCE
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;unistd.h&gt;
#include &lt;semaphore.h&gt;
#include &lt;sys/mman.h&gt;
#include &lt;sys/wait.h&gt;
#include &lt;time.h&gt;
#define CH 65536
#define SLOTS 8
typedef struct { sem_t empty, full; size_t len[SLOTS]; char buf[SLOTS][CH]; } Ring;
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&amp;t);return t.tv_sec+t.tv_nsec*1e-9;}
static unsigned long sum(const char*p,size_t n){unsigned long s=0;for(size_t i=0;i&lt;n;i++)s+=(unsigned char)p[i];
return s;}
int main(int argc,char**argv){
    const char*mode=argc&gt;1?argv[1]:"pipe"; size_t total=(size_t)(argc&gt;2?atoi(argv[2]):1)&lt;&lt;20;
    static char src[CH]; for(size_t i=0;i&lt;CH;i++) src[i]=(char)(i*7);
    unsigned long expect=0; for(size_t o=0;o&lt;total;o+=CH) expect+=sum(src,CH);
    double t0=now();
    if(!strcmp(mode,"pipe")){
        int fd[2]; pipe(fd);
        if(fork()==0){ close(fd[1]); static char b[CH]; unsigned long s=0; size_t got=0; ssize_t k;
            while((k=read(fd[0],b,CH))&gt;0){ s+=sum(b,k); got+=k; }
            printf("consumer: %zu bytes, checksum %s\n",got,s==expect?"OK":"MISMATCH"); fflush(stdout); _exit(0);}
        close(fd[0]); for(size_t o=0;o&lt;total;o+=CH) write(fd[1],src,CH);
        close(fd[1]); wait(NULL);
    } else {
        Ring*r=mmap(NULL,sizeof(Ring),PROT_READ|PROT_WRITE,MAP_SHARED|MAP_ANONYMOUS,-1,0);
        sem_init(&amp;r-&gt;empty,1,SLOTS); sem_init(&amp;r-&gt;full,1,0);          /* pshared = 1 */
        if(fork()==0){ unsigned long s=0; size_t got=0; int i=0;
            for(;;){ sem_wait(&amp;r-&gt;full); size_t n=r-&gt;len[i]; if(n==0) break;   /* len 0 = end of data */
                s+=sum(r-&gt;buf[i],n); got+=n; sem_post(&amp;r-&gt;empty); i=(i+1)%SLOTS; }
            printf("consumer: %zu bytes, checksum %s\n",got,s==expect?"OK":"MISMATCH"); fflush(stdout); _exit(0);}
        int i=0; for(size_t o=0;o&lt;total;o+=CH){ sem_wait(&amp;r-&gt;empty); memcpy(r-&gt;buf[i],src,CH); r-&gt;
len[i]=CH; sem_post(&amp;r-&gt;full); i=(i+1)%SLOTS; }
        sem_wait(&amp;r-&gt;empty); r-&gt;len[i]=0; sem_post(&amp;r-&gt;full); wait(NULL);
    }
    printf("%-4s %4zu MB  %.4f s  %.0f MB/s\n",mode,total&gt;&gt;20,now()-t0,(total&gt;&gt;20)/(now()-t0));
    return 0; }
```

5. Measure: transfer of 1 MB, 10 MB and 100 MB

Median of 5 runs; both methods include the same consumer checksum work, and the checksum was **OK for every run** (no data lost). Script: `bench.sh`.

Size	Pipe (s)	Pipe MB/s	Shared memory + semaphores (s)	Shm MB/s	Shm faster by
1 MB	0.0014	714	0.0016	625	0.88×
10 MB	0.0074	1351	0.0063	1587	1.17×
100 MB	0.1437	696	0.0636	1572	2.26×



6. Evaluate and compare

Growth: time grows with size and is O(n) in theory. Measured, small runs are dominated by fixed costs, so growth is not exactly 10× per step: from 10 MB to 100 MB the pipe time rises about 19× (0.0074 → 0.144 s, ≈ 700 MB/s) and shared memory about 10× (0.0063 → 0.064 s, ≈ 1,570 MB/s). The pipe's extra slowdown at 100 MB is consistent with its two copies per byte, though I did not profile the cause. At **1 MB and 10 MB the difference is small** (within about 1.2× and within run-to-run noise): fixed costs (process creation, setup, scheduling) dominate such short runs. At **100 MB shared memory is about 2.3× faster**, since it avoids the two kernel copies that the pipe makes for every byte, so its advantage grows with data size.

Why shared memory is not infinitely faster: the producer still writes each byte once (memcpy into the ring), the consumer still reads each byte once (checksum), and every 64 KB slot costs two semaphore operations (sem_wait/sem_post, potentially a futex system call when blocking). A pipe needs one write() and one read() system call per chunk plus the two copies.

Criterion	Pipe	Shared memory + semaphores
Model	Message passing (byte stream, kernel buffer)	Shared memory
Copies of the data	2 through the kernel per transfer	0 kernel copies (data stays in the shared pages)
Extra synchronisation	None (kernel blocks readers/writers)	Required: semaphores/mutex (race risk if wrong)
End of data	Writer closes → read() returns 0	Must be defined by the programmer (here slot length 0)
Ease and safety	Simple; hard to misuse	Complex; deadlock/race bugs possible
Speed on 100 MB (measured)	0.144 s	0.064 s
Works between unrelated processes	Anonymous pipe: only related processes (named FIFO for unrelated)	Yes, with named shm (shm_open) or inherited mapping

7. Alternative concept: shared memory + semaphore instead of a pipe – complexity

Solution (bounded buffer, as in ipc.c):

```
producer:  for each reading r:  sem_wait(empty); buf[in] = r; in = (in+1) % K; sem_post(full)
```

```
consumer:  for each reading:     sem_wait(full);  r = buf[out]; out = (out+1) % K; sem_post(empty)
init:      empty = K, full = 0; ring lives in shared memory (1 producer + 1 consumer: no mutex needed)
```

For n readings of size s	Pipe	Shared memory + semaphore
Time	O(n·s) data copying: 2 copies (user→kernel→user) = 2n·s bytes moved by the kernel, plus ≈ n·s/chunk system calls	O(n·s) at most one write by producer and one read by consumer, no kernel copy; ≈ 2 semaphore ops per slot (system calls only when blocking)
Space	O(pipe buffer) = 64 KB by default in the kernel + user buffers	O(K·slot size) shared region, fixed and chosen by the programmer
Synchronisation	Implicit in read()/write() blocking	Explicit semaphores (empty/full) – extra code and risk

Both are linear in n; the shared-memory solution has a smaller constant (no kernel copying), which shows up clearly at 100 MB (2.3×). For a single weather station sending a few bytes per second, the difference is invisible.

8. Design decision

Choose a pipe (or a message queue) for the weather station. Sensor readings are small and arrive slowly (a few hundred bytes per second), so the copy overhead is irrelevant, whereas a pipe gives built-in blocking synchronisation, no lost readings and a clear end-of-data signal with no extra code or locking bugs. **Use shared memory with semaphores only if the data rate becomes large** (for example images or raw high-frequency waveforms of tens to hundreds of MB), where the measured 2.3× advantage is worth the extra complexity. In both cases the unused pipe ends must be closed, otherwise the analysis process never receives end of data.

Limitations

Timings come from a 2-core virtualised container, so absolute speeds will differ on your machine; small-size runs (1 MB, 10 MB) are noisy. I implemented the pipe and shared-memory methods; a POSIX message queue was not implemented since the task allows either one. Run the programs yourself, commit `Task04/pipe_demo.c, ipc.c, bench.sh` and outputs, and add your name, register number and GitHub link.

Operating Systems Assignment 2 – Task 5 (Q5)

Synchronisation: Critical Sections, Semaphores and Monitors – Railway Ticket Booking (CO2, K5 – Evaluate, 11 marks). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0, 2 CPUs. Compile: `gcc -std=c11 -O1 -Wall -Wextra -pthread`. Structure follows the expected flow *Model* → *Design* → *Implement* → *Measure* → *Compare* → *Evaluate* → *Recommend*.

1. Model: the problem

Thousands of agents book seats at once. The seat map is shared data. Two agents can both read “seat 12 is free” and both sell it (a **race condition**) unless the check-and-update is a **critical section**. A single global lock removes the race but serialises every booking.

2. Design: from the principle to a solution

Principle given: a critical-section solution must guarantee (1) **mutual exclusion** – at most one agent updates a seat at a time; (2) **progress** – if nobody is in the critical section, an agent that wants in is not blocked by agents outside it, and the decision cannot be postponed forever; (3) **bounded waiting** – a limit on how many others can enter before a waiting agent. My design turns these into rules:

Requirement	Design rule for the booking system
Mutual exclusion	Check-free-then-mark-sold on a seat is one atomic section guarded by a lock that covers that seat.
Progress	Locks are held only inside the section (never while waiting for the customer); agents booking different seats never block each other.
Bounded waiting	Use fair/blocking primitives (mutex or semaphore queue); no agent can be overtaken indefinitely, and critical sections are short.
Performance	Use the finest lock granularity whose bookkeeping is still simple (Section 8).

3. Peterson's solution: why it works for two processes, and why it may fail today

```
shared: flag[2] = {false,false}; turn
Pi:  flag[i] = true;  turn = j;
     while (flag[j] && turn == j) ;      // busy wait
     ... critical section ...
     flag[i] = false;
```

Requirement	Argument
Mutual exclusion	Pi enters only if flag[j]==false or turn==i. If both were inside, both flags are true, so turn==i and turn==j would both have to hold, which is impossible (turn is a single variable and each process sets it to the *other's* id before waiting). So the one that wrote turn last waits.
Progress	If Pj is not interested, flag[j]==false and Pi enters at once. If both want in, turn decides within one step.
Bounded waiting	After leaving, Pj sets flag[j]=false; if it comes back it sets turn=i, giving Pi priority, so Pi waits for at most one entry of Pj.

Why it fails on modern processors: the proof assumes loads and stores happen in program order. Modern CPUs and compilers reorder them (store buffers let the load of `flag[j]` complete before the store to `flag[i]` is visible to the other core), so both threads may see the other's flag as false and both enter. Measured (`peterson.c`, 2 threads × 2,000,000 increments):

```
plain  expected 4000000  got 3988839  MUTUAL EXCLUSION VIOLATED
plain  expected 4000000  got 3993753  MUTUAL EXCLUSION VIOLATED
plain  expected 4000000  got 3984337  MUTUAL EXCLUSION VIOLATED
fenced expected 4000000  got 4000000  mutual exclusion held
fenced expected 4000000  got 4000000  mutual exclusion held
fenced expected 4000000  got 4000000  mutual exclusion held
```

With ordinary variables roughly 6,000–16,000 of 4,000,000 updates were lost in each run (mutual exclusion violated). With sequentially consistent atomics (which insert memory fences) the result is exact. In practice use mutexes or atomics rather than Peterson; also busy-waiting wastes CPU.

4. Semaphores protecting the seat counter

```
wait(S):    S = S - 1;  if (S < 0) block the caller on S's queue     (atomic)
signal(S):  S = S + 1;  if (S <= 0) wake one blocked process           (atomic)

Counting semaphore  seats = number of free seats (e.g. 40):
    book():    wait(seats);     // blocks when 0 seats are left (or the booking is refused with try_wait)
    cancel():  signal(seats);

Binary semaphore  mutex = 1  (protects the seat map / counter update):
    wait(mutex);  mark seat sold; sold++;  signal(mutex);
```

The counting semaphore keeps the number of sold seats ≤ capacity (a resource count); the binary semaphore gives mutual exclusion over the data structure. Blocked processes sleep in the semaphore's queue (no CPU), and a FIFO queue gives bounded waiting.

5. Bounded buffer, readers–writers, dining philosophers with semaphores

5.1 Bounded buffer (producer–consumer)

```
semaphore empty = N, full = 0, mutex = 1
producer: wait(empty); wait(mutex); buf[in]=item; in=(in+1)%N; signal(mutex); signal(full)
consumer: wait(full);  wait(mutex); item=buf[out]; out=(out+1)%N; signal(mutex); signal(empty)
```

empty blocks producers when the buffer is full, **full** blocks consumers when it is empty, and **mutex** protects the in/out indices when there are several producers or consumers. Order matters: wait on the counting semaphore *before* the mutex, otherwise a producer holding the mutex while waiting for a free slot deadlocks the consumers.

5.2 Readers–writers (readers' priority)

```
semaphore rw = 1, mutex = 1;  int readcount = 0
reader: wait(mutex); readcount++; if (readcount==1) wait(rw); signal(mutex);
        ... read ...
        wait(mutex); readcount--; if (readcount==0) signal(rw); signal(mutex)
writer: wait(rw);  ... write ...;  signal(rw)
```

Many readers (agents just viewing seat availability) share access; a writer (booking) gets exclusive access. Weakness: a continuous stream of readers can starve writers; the fix is a writers'-priority or fair (turnstile) variant.

5.3 Dining philosophers

Five philosophers, one semaphore per fork. If everyone picks up the left fork then the right, all hold one fork and wait forever (deadlock, circular wait). Fix used here: resource ordering – pick up the lower-numbered fork first, which breaks the circular wait (other fixes: allow at most 4 at the table; pick both forks inside a critical section). Program `dining.c`:

```
ordered: all philosophers finished, meals = 200 200 200 200 200
ordered exit=0
naive exit=124 (124 = hung: deadlock, killed by timeout)
```

The naive version hangs (killed by timeout after 5 s); the ordered version completes all 200 meals for each philosopher.

6. Monitors and condition variables

A monitor bundles the shared data, the lock and the waiting conditions in one construct: only one thread is active inside at a time, and `wait(cond)` releases the monitor lock and sleeps atomically; `signal(cond)` wakes a waiter. This is easier to get right than raw semaphores because there is no separate empty/full/mutex ordering to get wrong; the waiting condition is written in plain code (`while (count==N) wait(not_full)`).

```
put(x):  lock(m); while (count == N) cond_wait(not_full, m);
         buf[in]=x; in=(in+1)%N; count++; cond_signal(not_empty); unlock(m)
get():   lock(m); while (count == 0) cond_wait(not_empty, m);
         x=buf[out]; out=(out+1)%N; count--; cond_signal(not_full); unlock(m); return x
```

Implemented in `monitor.c` with pthread mutex + condition variables. 4 producers + 4 consumers, 100 runs:

```
monitor 4x4: 100/100 correct
```

7. Test the design: bounded buffer program (Complete the Code, TODO 1 and TODO 2)

TODO 1 (producer): `sem_wait(&empty)`, lock mutex, insert, unlock, `sem_post(&full)`. TODO 2 (consumer): `sem_wait(&full)`, lock mutex, remove, unlock, `sem_post(&empty)`. Items 1..20 are produced, so the sum must be 210 and every item must be consumed exactly once (checked with a per-item counter).

```
/* Bounded buffer (producer-consumer) with counting semaphores + mutex.
   Items 1..20 are produced; the sum of consumed items must be 210 and every item consumed exactly once.
   gcc -std=c11 -O1 -Wall -Wextra -pthread -o bb bb.c
   ./bb &lt;producers&gt; &lt;consumers&gt; [nomutex]     (producers and consumers must divide 20) */
#define _GNU_SOURCE
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;pthread.h&gt;
#include &lt;semaphore.h&gt;
#include &lt;stdatomic.h&gt;
#include &lt;sched.h&gt;
#define BUF 5
#define ITEMS 20
static int buf[BUF], in = 0, out = 0, P, C, use_mutex = 1;
static sem_t empty_s, full_s;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static atomic_long sum; static atomic_int seen[ITEMS + 1];
#define LOCK()   do { if (use_mutex) pthread_mutex_lock(&amp;m); } while (0)
#define UNLOCK() do { if (use_mutex) pthread_mutex_unlock(&amp;m); } while (0)
static void *producer(void *arg) {
    int p = (int)(long)arg;
    for (int k = 1; k &lt;= ITEMS; k++) {
        if ((k - 1) % P != p) continue;              /* this producer's share of items */
        sem_wait(&amp;empty_s);                          /* TODO 1 (completed): wait for a free slot   */
        LOCK();                                      /*   mutex protects in/out indices            */
        int slot = in; sched_yield(); buf[slot] = k; in = (slot + 1) % BUF;
        UNLOCK();
        sem_post(&amp;full_s);                           /*   signal: one more full slot               */
    }
    return NULL;
}
static void *consumer(void *arg) {
    (void)arg;
    for (int n = 0; n &lt; ITEMS / C; n++) {
        sem_wait(&amp;full_s);                           /* TODO 2 (completed): wait for a full slot   */
        LOCK();
        int slot = out; sched_yield(); int item = buf[slot]; out = (slot + 1) % BUF;
        UNLOCK();
        sem_post(&amp;empty_s);                          /*   signal: one more free slot               */
        atomic_fetch_add(&amp;sum, item); atomic_fetch_add(&amp;seen[item], 1);
    }
    return NULL;
}
int main(int argc, char **argv) {
    P = argc &gt; 1 ? atoi(argv[1]) : 1; C = argc &gt; 2 ? atoi(argv[2]) : 1;
    if (argc &gt; 3 &amp;&amp; !strcmp(argv[3], "nomutex")) use_mutex = 0;
    if (ITEMS % P || ITEMS % C) { fprintf(stderr, "P and C must divide %d\n", ITEMS); return 2; }
    sem_init(&amp;empty_s, 0, BUF); sem_init(&amp;full_s, 0, 0);
    pthread_t tp[P], tc[C];
    for (long i = 0; i &lt; P; i++) pthread_create(&amp;tp[i], NULL, producer, (void *)i);
    for (long i = 0; i &lt; C; i++) pthread_create(&amp;tc[i], NULL, consumer, NULL);
    for (int i = 0; i &lt; P; i++) pthread_join(tp[i], NULL);
    for (int i = 0; i &lt; C; i++) pthread_join(tc[i], NULL);
    int once = 1; for (int k = 1; k &lt;= ITEMS; k++) if (seen[k] != 1) once = 0;
    printf("P=%d C=%d mutex=%s  sum=%ld  each item exactly once: %s\n", P, C, use_mutex ? "yes" : "NO", (long)sum,
once ? "yes" : "NO");
    return (sum == 210 &amp;&amp; once) ? 0 : 1;
}
```

Single runs

```
$ ./bb 1 1
P=1 C=1 mutex=yes  sum=210  each item exactly once: yes
$ ./bb 4 4
P=4 C=4 mutex=yes  sum=210  each item exactly once: yes
```

100 runs each, with and without the mutex (run100.sh)

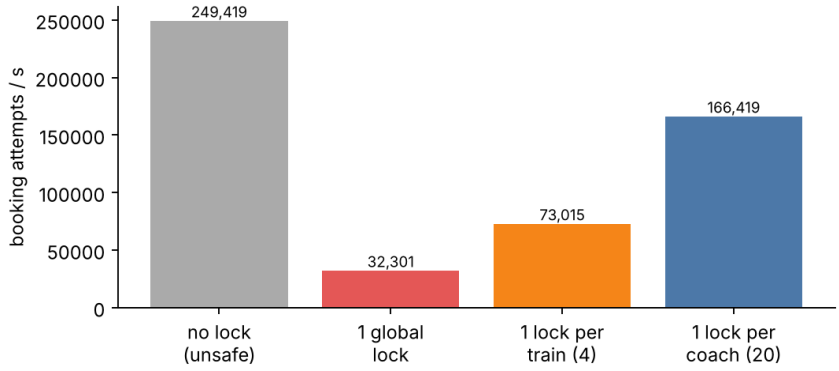
```
args: 1 1  -&gt;  correct runs: 100 / 100   incorrect: 0
args: 4 4  -&gt;  correct runs: 100 / 100   incorrect: 0
args: 1 1 nomutex  -&gt;  correct runs: 100 / 100   incorrect: 0
args: 4 4 nomutex  -&gt;  correct runs: 0 / 100   incorrect: 100
example incorrect run: P=4 C=4 mutex=NO  sum=197  each item exactly once: NO
```

With the mutex, 1×1 and 4×4 were correct in 100 of 100 runs (sum always 210, each item exactly once). **Without the mutex:** with 1 producer and 1 consumer it is still correct (100/100), because only one thread ever changes `in` and only one changes `out`, and the semaphores already order the hand-over. With 4 producers and 4 consumers it was **wrong in 100 of 100 runs** (example: sum 197): two producers read the same `in` and overwrite each other's item, and two consumers read the same `out` and take the same item, so some items are lost and some consumed twice. (To make the race show reliably I put a `sched_yield()` between reading and using the index; without it the race exists but appears less often.)

8. Deployment decision: one lock for the system, per train or per coach

`booking.c`: 4 trains × 5 coaches × 40 seats = 800 seats; 8 agent threads × 400 random booking attempts; a booking checks the seat, then performs a 50 µs payment/database write, then marks it sold. Median of 3 runs; “seats sold twice” counts duplicates.

Strategy	Locks	Time (s)	Attempts / s	Speed-up vs global	Seats sold twice
No lock	0	0.013	249,419	7.72×	[27, 22, 19]
One global lock	1	0.099	32,301	1.00×	[0, 0, 0]
One lock per train	4	0.044	73,015	2.26×	[0, 0, 0]
One lock per coach	20	0.019	166,419	5.15×	[0, 0, 0]



Evaluation: without locks the booking is fastest but sold seats twice (19–27 duplicates per run) – unacceptable. The global lock is correct but only ≈ 32,000 attempts/s because all 8 agents queue behind one another's 50 µs write. A lock per train allows 4 bookings in parallel (2.3×); a lock per coach allows up to 20 (5.1×). All three locking strategies have zero double sales and the same number of sales (781).

Decision: one lock per coach (each coach's seat map has its own mutex). It is correct, gives the highest measured throughput, and bookings for different coaches or trains never wait for each other. Costs: 20 locks instead of 1 (small memory), and any operation that touches several coaches (e.g. booking a family across two coaches, or a train-wide report) must acquire the locks in a fixed order (coach id) to avoid deadlock; those rarer operations can take the per-train lock set in order. Per-coach is not unlimited: the gain is limited by the number of agents and by how many requests target the same coach (popular coaches become hot spots).

9. Alternative concept: atomic operations or a spin lock instead of semaphore + mutex

Solution: for a plain counter, `atomic_fetch_add(&sold, 1)` (C11) performs the read-modify-write as one indivisible hardware instruction (no lock, no waiting queue). A spin lock replaces the sleeping lock by a busy loop: `while (atomic_flag_test_and_set(&l)) ; counter++; atomic_flag_clear(&l);`. Measured with `counter.c` (500,000 updates per thread; time per update = wall time ÷ total updates; CPU time includes all threads):

Method	Threads	ns per update	CPU time (s)	Result
mutex	1	16.9	0.009	correct
mutex	4	49.4	0.156	correct
mutex	16	53.6	0.780	correct

Method	Threads	ns per update	CPU time (s)	Result
sem	1	21.8	0.012	correct
sem	4	197.5	0.781	correct
sem	16	223.9	3.501	correct
spin	1	9.8	0.006	correct
spin	4	80.3	0.314	correct
spin	16	473.7	7.545	correct
atomic	1	6.2	0.004	correct
atomic	4	30.2	0.118	correct
atomic	16	32.2	0.496	correct

Time per update: with 1 thread (no contention) atomic ≈ 6 ns, spin 10 ns, mutex 17 ns, semaphore 22 ns. Under contention (4 and 16 threads on only 2 cores) the atomic counter stays at ≈ 30 ns, the mutex ≈ 50 ns, the semaphore ≈ 200 ns, and the spin lock degrades badly (80 ns with 4, 474 ns with 16 threads). **CPU wasted by spinning:** with 16 threads the spin lock used 7.5 s of CPU for 8 million updates, 15× the atomic counter (0.50 s) and 10× the mutex (0.78 s); a thread that is spinning when the lock holder is pre-empted burns its whole time slice doing nothing useful. All methods updated the counter correctly. Complexity: each update is O(1) for all methods; under contention the expected waiting grows with the number of contenders p, O(p) for lock-based methods.

Long critical section (4 threads, each update holds the lock for a 50 µs wait, 300 updates each):

```
--- long critical section (50 us wait inside, 4 threads x 300 updates)
method threads ns_per_update cpu_seconds correct
mutex 4 129505.0 0.034 ok
sem 4 123307.3 0.033 ok
spin 4 131838.9 0.277 ok
```

Elapsed time per update is the same (≈ 125–130 µs, since the critical sections are serialised anyway) but the spin lock used **0.277 s of CPU against 0.033 s for the mutex and 0.034 s for the semaphore** (≈ 8×): spinners burn CPU while the holder is waiting. **So spin locks suit only very short critical sections (a few instructions) on multicore machines, atomics suit single-variable updates, and mutexes/semaphores (sleeping locks) suit long critical sections** such as a booking with a payment or database write.

10. Final recommendation

Use **one mutex per coach** to protect each seat map (measured 5.1× the throughput of one global lock with no double sales), a **counting semaphore or atomic counter** for available-seat counts, a monitor (mutex + condition variables) wherever agents must wait for a condition such as “a seat becomes free”, and fixed lock ordering when an operation needs several locks. Do not use Peterson's algorithm or spin locks on production booking paths.

Limitations

Timings were taken on a 2-core virtualised container and the booking test has only 3 short repetitions (consistent within ≈ 10%); absolute numbers will differ on your machine. The 50 µs payment delay is simulated. Peterson's failure depends on the CPU's memory model (shown here on x86). Run the programs yourself, commit `Task05/*.c, run100.sh` and outputs, and add your name, register number and GitHub link.

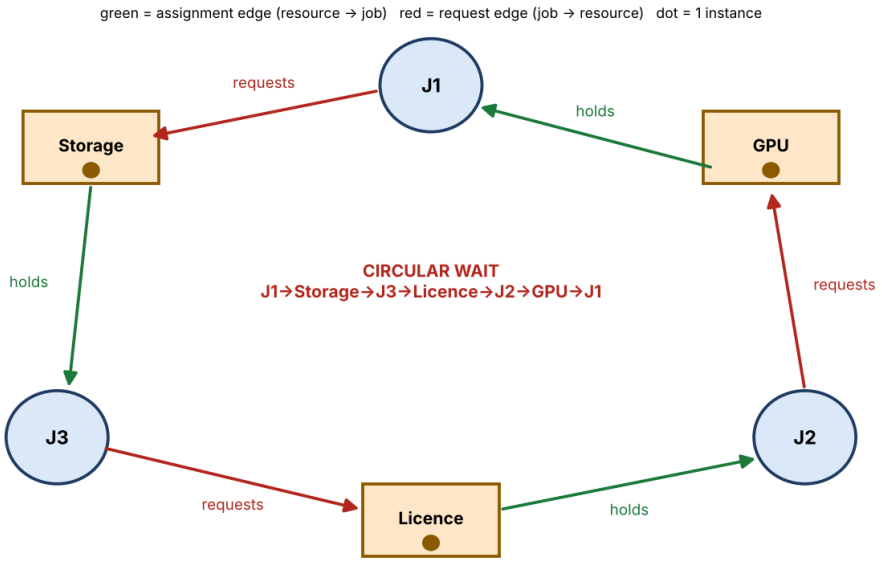
Operating Systems Assignment 2 – Task 6 (Q6, Day-4 Design Challenge)

Deadlock Handling and the Banker's Algorithm – Shared Laboratory Equipment Scheduler (CO2, K5 – Evaluate, 13 marks). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0. Compile: `gcc -std=c11 -O2 -Wall -Wextra`. The sections below follow the marking parts of the question: Characterise (2), Compare (3), Apply (4), Predict (1), Measure (1), Defend (2).

1. Characterise deadlock (2 marks)

Context: jobs need several shared resources at once (GPUs, simulation-software licences, high-speed storage). A deadlock occurs when a set of jobs each hold something and wait for something held by another job in the set. **Four necessary conditions** (all four must hold together):

Condition	Meaning	Where it appears in the cluster
1. Mutual exclusion	A resource can be used by only one job at a time	A GPU, a licence seat, or a storage lane is exclusive while in use
2. Hold and wait	A job holds at least one resource while waiting for another	J1 holds a GPU and waits for storage; J2 holds the licence and waits for a GPU
3. No pre-emption	A resource cannot be forcibly taken from a job	A running simulation cannot lose its licence or GPU without losing its work
4. Circular wait	A cycle of jobs, each waiting for a resource held by the next	J1 → Storage → J3 → Licence → J2 → GPU → J1



Resource-allocation graph. Squares are resource types (one instance each), circles are jobs. A green edge resource → job means the resource is allocated to the job; a red edge job → resource means the job is waiting for it. Every resource has one instance, so the cycle J1 → Storage → J3 → Licence → J2 → GPU → J1 *is* a deadlock: no job can proceed and none will release what it holds. Condition 1 is the single-instance resources, 2 is the pair of green and red edges at every job, 3 is the fact that green edges cannot be removed by force, and 4 is the cycle.

2. Compare the four strategies for the cluster (3 marks)

Strategy	How it works	Advantages	Disadvantages for this cluster
Prevention (remove one condition)	Hold-and-wait: request all resources at once before starting. Circular wait: number the resources (e.g. storage < licence < GPU) and request in order. No pre-emption: allow forced release (checkpoint and restart). Mutual exclusion cannot be removed for GPUs and licences.	Simple, no run-time checking, deadlock impossible	All-at-once allocation leaves expensive GPUs idle while the job waits for its licence (low utilisation) and may starve large jobs; ordering needs every job to follow it; pre-emption loses work
Avoidance (Banker's algorithm)	Each job declares its maximum demand in advance; the scheduler grants a request only if the resulting state is safe (a safe sequence exists)	No deadlock, better utilisation than all-at-once (resources are given incrementally), no jobs aborted	Needs maximum demands up front and a fixed set of resources; $O(m \cdot n^2)$ per check; conservative (may delay safe requests)
Detection and recovery	Allow deadlock; run a detection algorithm periodically; recover by aborting a victim job (or pre-empting a resource)	Maximum utilisation, no advance information, no per-request cost	Deadlocked jobs waste resources until the next detection run; aborting loses work (hours for a long simulation); needs checkpointing to be cheap
Ignore (ostrich)	Assume deadlock is rare; an administrator restarts the cluster	Zero cost and zero complexity	This cluster already deadlocks and stops until an administrator intervenes; unacceptable for expensive, long-running jobs

Conclusion of the comparison: ignoring deadlock is what the centre has now and is the problem. Prevention by all-at-once allocation wastes expensive GPUs. Avoidance fits a batch cluster well because jobs already declare their resource needs when submitted, and detection is a cheap safety net for what avoidance cannot see (see the final recommendation).

3. Apply the Banker's algorithm (4 marks)

Data. The assignment text does not list the allocation tables, so I used the standard textbook state: 5 jobs P0–P4, resource types A, B, C with totals (10, 5, 7); the question's request “P0 asks for (0,2,0)” and the 3 resource types match this standard example (A = GPUs, B = licences, C = storage lanes in the cluster story). If your own state differs, replace the matrices in `banker.c` and rerun.

Algorithms. Safety: Work = Available, Finish = false; repeatedly find an unfinished job with Need ≤ Work, add its Allocation to Work and mark it finished; the state is safe if all jobs finish. Resource-request: for request R from Pi: (1) if R > Need[i] error; (2) if R > Available, wait; (3) pretend to allocate (Available –= R, Alloc[i] += R, Need[i] –= R); run the safety algorithm; if safe, grant, otherwise restore the old state and make Pi wait.
Program: TODO 1 is the “can this job finish?” test inside the safety loop.

```
/* Banker's algorithm: safety algorithm + resource-request algorithm, and a timing benchmark.
   gcc -std=c11 -O2 -Wall -Wextra -o banker banker.c
   ./banker            -&gt; run the given state (5 processes, resources A,B,C) and the requests
   ./banker bench n m  -&gt; time the safety algorithm on a worst-case state with n processes, m resource types */
#define _POSIX_C_SOURCE 200809L
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;time.h&gt;
#define MAXN 5000
#define MAXM 16
typedef struct { int n, m; int avail[MAXM]; int max[MAXN][MAXM], alloc[MAXN][MAXM], need[MAXN][MAXM]; } State;
static State S;
/* Safety algorithm. Returns 1 and fills seq[] if the state is safe. Cost O(m * n^2). */
static int is_safe(const State *s, int *seq) {
    int work[MAXM]; static int finish[MAXN]; int count = 0;
    memcpy(work, s-&gt;avail, sizeof(int) * s-&gt;m);
    memset(finish, 0, sizeof(int) * s-&gt;n);
    while (count &lt; s-&gt;n) {
        int found = 0;
        for (int i = 0; i &lt; s-&gt;n; i++) {
            if (finish[i]) continue;
            /* TODO 1 (completed): can process i finish?  need[i] &lt;= work (every resource type) */
            int ok = 1;
            for (int j = 0; j &lt; s-&gt;m; j++) if (s-&gt;need[i][j] &gt; work[j]) { ok = 0; break; }
            if (ok) {                                   /* i finishes and returns its allocation */
```

```
                for (int j = 0; j &lt; s-&gt;m; j++) work[j] += s-&gt;alloc[i][j];
                finish[i] = 1; seq[count++] = i; found = 1;
            }
        }
        if (!found) return 0;                           /* nobody can finish: unsafe */
    }
    return 1;
}
/* Resource-request algorithm: 1 = granted, 0 = must wait/denied (reason printed). */
static int request(State *s, int p, const int *req, const char *label) {
    int seq[MAXN];
    printf("\nRequest: P%d asks for %s\n", p, label);
    for (int j = 0; j &lt; s-&gt;m; j++) if (req[j] &gt; s-&gt;need[p][j]) { printf("  DENIED: exceeds its
declared maximum claim (error)\n"); return 0; }
    for (int j = 0; j &lt; s-&gt;m; j++) if (req[j] &gt; s-&gt;avail[j]) { printf("  WAIT: not enough resources
available\n"); return 0; }
    State t = *s;                                       /* pretend to allocate */
    for (int j = 0; j &lt; s-&gt;m; j++) { t.avail[j] -= req[j]; t.alloc[p][j] += req[j]; t.need[p][j] -= req[j]; }
    if (is_safe(&amp;t, seq)) {
        *s = t; printf("  GRANTED: new state is safe, safe sequence &lt;");
        for (int i = 0; i &lt; s-&gt;n; i++) printf("P%d%s", seq[i], i &lt; s-&gt;n - 1 ? ", " : "");
        printf("&gt;\n"); return 1;
    }
    printf("  DENIED: granting it would leave an UNSAFE state (no process could finish), request must wait\n");
    return 0;
}
static void show(const State *s) {
    printf("      Alloc   Max     Need\n");
    for (int i = 0; i &lt; s-&gt;n; i++) {
        printf("  P%d  ", i);
        for (int j = 0; j &lt; s-&gt;m; j++) { printf("%d ", s-&gt;alloc[i][j]); }
        printf("  ");
        for (int j = 0; j &lt; s-&gt;m; j++) { printf("%d ", s-&gt;max[i][j]); }
        printf("  ");
        for (int j = 0; j &lt; s-&gt;m; j++) { printf("%d ", s-&gt;need[i][j]); }
        printf("\n");
    }
    printf("  Available: ");
    for (int j = 0; j &lt; s-&gt;m; j++) { printf("%d ", s-&gt;avail[j]); }
    printf("\n");
}
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &amp;t); return t.tv_sec + t.tv_nsec *
1e-9; }
int main(int argc, char **argv) {
    if (argc &gt; 1 &amp;&amp; !strcmp(argv[1], "bench")) {
        int n = atoi(argv[2]), m = atoi(argv[3]); static int seq[MAXN];
        S.n = n; S.m = m;                               /* worst case: jobs finish only in reverse order; last
resource decides, so all m are compared */
        for (int j = 0; j &lt; m; j++) S.avail[j] = 1;
        for (int i = 0; i &lt; n; i++) for (int j = 0; j &lt; m; j++) { S.alloc[i][j] = 1; S.need[i][j] = (j == m -
 1) ? n - i : 1; S.max[i][j] = S.need[i][j] + 1; }
        int reps = (int)(2e8 / ((double)n * n * m / 2 + 1)); if (reps &lt; 3) reps = 3; if (reps &gt; 200000) reps
= 200000;
        int ok = 0; double t0 = now();
        for (int r = 0; r &lt; reps; r++) ok = is_safe(&amp;S, seq);
        double per = (now() - t0) / reps;
        printf("n=%-5d m=%-3d safe=%d  time per safety check = %.3f us  (%d reps)\n", n, m, ok, per * 1e6, reps);
        return 0;
    }
    int A[5][3] = {{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}}, M[5][3] = {{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}},
tot[3] = {10,5,7};
    S.n = 5; S.m = 3;
    for (int j = 0; j &lt; 3; j++) { int used = 0; for (int i = 0; i &lt; 5; i++) used += A[i][j]; S.avail[j] =
tot[j] - used; }
    for (int i = 0; i &lt; 5; i++) for (int j = 0; j &lt; 3; j++) { S.alloc[i][j] = A[i][j]; S.max[i][j] = M[i][j];
 S.need[i][j] = M[i][j] - A[i][j]; }
    printf("Total resources A,B,C = 10,5,7\nInitial state:\n"); show(&amp;S);
    int seq[5]; printf("\nSafety algorithm on initial state: ");
    if (is_safe(&amp;S, seq)) { printf("SAFE, sequence &lt;"); for (int i = 0; i &lt; 5; i++) printf("P%d%s", seq[
i], i &lt; 4 ? ", " : ""); printf("&gt;\n"); } else printf("UNSAFE\n");
    State orig = S;
    int r1[3] = {0,2,0}; request(&amp;S, 0, r1, "(0,2,0)  [evaluated alone, on the initial state]");
    S = orig;
    int r2[3] = {1,0,2}; request(&amp;S, 1, r2, "(1,0,2)");
    int r3[3] = {3,3,0}; request(&amp;S, 4, r3, "(3,3,0)");
    int r4[3] = {0,2,0}; request(&amp;S, 0, r4, "(0,2,0)  [after P1's (1,0,2) was granted]");
    return 0;
}
```

Execution output

```
Total resources A,B,C = 10,5,7
Initial state:
```

```
      Alloc   Max     Need
  P0  0 1 0   7 5 3   7 4 3
  P1  2 0 0   3 2 2   1 2 2
  P2  3 0 2   9 0 2   6 0 0
  P3  2 1 1   2 2 2   0 1 1
  P4  0 0 2   4 3 3   4 3 1
  Available: 3 3 2

Safety algorithm on initial state: SAFE, sequence &lt;P1, P3, P4, P0, P2&gt;

Request: P0 asks for (0,2,0)  [evaluated alone, on the initial state]
  GRANTED: new state is safe, safe sequence &lt;P3, P1, P2, P0, P4&gt;

Request: P1 asks for (1,0,2)
  GRANTED: new state is safe, safe sequence &lt;P1, P3, P4, P0, P2&gt;

Request: P4 asks for (3,3,0)
  WAIT: not enough resources available

Request: P0 asks for (0,2,0)  [after P1's (1,0,2) was granted]
  DENIED: granting it would leave an UNSAFE state (no process could finish), request must wait
```

Safe sequence of the initial state: <P1, P3, P4, P0, P2> (the sequence is not unique; <P1, P3, P4, P2, P0> is also valid). Check by hand: Work = (3,3,2); P1 needs (1,2,2) ≤ Work → Work (5,3,2); P3 needs (0,1,1) → (7,4,3); P4 needs (4,3,1) → (7,4,5); P0 needs (7,4,3) → (7,5,5); P2 needs (6,0,0) → (10,5,7) = total. ✓

Request	Decision	Reason
P0 asks (0,2,0) on the initial state	Granted	≤ Need (7,4,3), ≤ Available (3,3,2); new state still safe, e.g. <P3, P1, P2, P0, P4>
P1 asks (1,0,2)	Granted	≤ Need (1,2,2), ≤ Available (3,3,2); safe sequence <P1, P3, P4, P0, P2>; Available becomes (2,3,0)
P4 asks (3,3,0)	Must wait	(3,3,0) is not ≤ Available (2,3,0): resources not available
P0 asks (0,2,0) after P1's request was granted	Denied (unsafe)	Passes the Need and Available checks but leaves no job able to finish (shown by hand below)

By hand: why P0's request for (0,2,0) leads to an unsafe state

Take the state after P1's request (1,0,2) was granted: Available = (2,3,0); Allocation P0 (0,1,0), P1 (3,0,2), P2 (3,0,2), P3 (2,1,1), P4 (0,0,2). Checks: request (0,2,0) ≤ Need[P0] (7,4,3) ✓ and ≤ Available (2,3,0) ✓. **Pretend to grant it:** Available = (2,1,0); Allocation P0 = (0,3,0); Need P0 = (7,2,3). The needs of all jobs are:

Job	Need (A,B,C)	Need ≤ Work (2,1,0)?	Why
P0	(7,2,3)	No	A: 7 > 2
P1	(0,2,0)	No	B: 2 > 1
P2	(6,0,0)	No	A: 6 > 2
P3	(0,1,1)	No	C: 1 > 0
P4	(4,3,1)	No	A: 4 > 2

No job can obtain its maximum remaining need, so none can run to completion and release resources: **there is no safe sequence** → **the state is unsafe**. It is not yet a deadlock (the jobs have not requested their maximum), but if the maximum demands were requested a deadlock would be possible, so the Banker's algorithm refuses the request and P0 waits. (Without P1's earlier grant the same request is safe, as the first row of the table shows: safety depends on the whole state.)

4. Predict before implementing (1 mark)

Prediction: each execution of the safety algorithm repeats up to n passes over the n jobs, and each job test compares up to m resource types, so the worst-case time is **T(n, m)** ≈ **c·m·n²**, O(m·n²). Testable consequences: (a) 10× more jobs → about **100×** more time; (b) doubling n → about **4×**; (c) time grows roughly linearly with m. I expected the measured ratios to fall a little below these because of fixed overhead per call and because the per-comparison cost is very small.

5. Measure (1 mark)

`./banker bench n m` times the safety algorithm on a constructed **worst-case** state (jobs can only finish in reverse order, and the last resource type decides, so all m resources are compared): many repetitions per size (mean time per call). Output:

```
n=10    m=3   safe=1  time per safety check = 0.220 us  (200000 reps)
n=100   m=3   safe=1  time per safety check = 12.979 us  (13332 reps)
n=1000  m=3   safe=1  time per safety check = 912.504 us  (133 reps)
n=2000  m=3   safe=1  time per safety check = 3930.011 us  (33 reps)
n=100   m=6   safe=1  time per safety check = 24.145 us  (6666 reps)
n=100   m=12  safe=1  time per safety check = 32.026 us  (3333 reps)
n=100   m=16  safe=1  time per safety check = 43.517 us  (2499 reps)
n=1000  m=12  safe=1  time per safety check = 3115.218 us  (33 reps)
```

Test	Predicted	Measured	Verdict
n: 10 → 100 (m=3)	≈ 100×	59× (0.22 → 12.979 µs)	Below prediction: at n=10 fixed call overhead is a large share of 0.2 µs
n: 100 → 1,000 (m=3)	≈ 100×	70× (12.979 → 912.504 µs)	Below prediction (CPU prediction/caching make large scans cheaper per step), same order of magnitude
n: 1,000 → 2,000 (m=3)	≈ 4×	4.3× (912.504 → 3930.011 µs)	Matches: quadratic growth confirmed
m: 3 → 12 at n=100	≈ 4× (linear in m)	2.5× (12.979 → 32.026 µs)	Matches roughly (a fixed part of the work does not depend on m)
m: 3 → 12 at n=1,000	≈ 4×	3.4×	Matches roughly

Conclusion: measured time grows quadratically in n (doubling n costs ≈ 4×) and roughly linearly in m, as predicted, i.e. $O(m \cdot n^2)$; for small n the ratios were lower than 100× because of constant overheads. Run-to-run variation of about ±20% was seen between two benchmark runs.

6. Alternative concept: detection and recovery instead of the Banker's algorithm

Solution: do not check requests; let jobs acquire resources freely. Every few minutes run a **detection algorithm** (same structure as the safety algorithm, but using the current pending *Request* matrix instead of Need, and not needing maximum demands): jobs that can never finish are deadlocked. **Recovery:** abort a victim (the job with least work done), release its resources, and repeat detection. `detect.c`, with the cluster state J0 holds GPU and wants licence, J1 holds licence and wants storage, J2 holds GPU and storage and wants a GPU (none left):

```
Detection run 1: 3 jobs deadlocked: J0 J1 J2
  recovery: abort victim J2 (least work done: 12 min lost)
Detection run 2: no deadlock
Total work lost by recovery: 12 job-minutes (J4 untouched, not deadlocked)
```

Aspect	Banker's algorithm (avoidance)	Detection and recovery
Time per request	$O(m \cdot n^2)$ on every request: measured 13 µs at n=100, 913 µs at n=1,000 (m=3)	None: requests are granted immediately (O(1))
Cost of the check	Paid per request: with 10,000 requests/hour at n=1,000 → ≈ 9.1 s of CPU per hour	Paid per detection run: every 5 min = 12 runs/hour, i.e. ≈ 11 ms of CPU per hour at n=1,000 (same $O(m \cdot n^2)$ cost per run)
Needs advance knowledge?	Yes: maximum demand of every job	No
Work lost	None (no aborts), but some requests wait longer	Victim's work since its last checkpoint is lost (12 job-minutes in the demo; hours for a long simulation), plus idle resources held by deadlocked jobs until the next run
Deadlock duration	Never occurs	Up to one detection interval (minutes) before it is found
Utilisation	Slightly reduced (safe-state requests only)	Highest until deadlock occurs

Note: in the demo, a job that holds nothing (J3) is waiting but cannot be part of a deadlock cycle, so the detector does not count it; J4 is not deadlocked and is untouched.

7. Defend the final design (2 marks)

Recommendation: use the Banker's algorithm (deadlock avoidance) inside the cluster scheduler for GPUs, licences and storage, with periodic deadlock detection and checkpoint-based recovery as a safety net. Justification:

Point	Evidence
Maximum demands are knowable	In a batch cluster every job is submitted with its declared resource needs (GPUs, licence seats, storage), so “maximum demand in advance” - the Banker's biggest weakness - is already part of the submission. The scheduler can kill or reject a job that exceeds its declared maximum.
The cost is negligible	One safety check costs 13 µs for 100 jobs and 913 µs for 1,000 jobs (measured, m=3), $O(m \cdot n^2)$ - against jobs that run for minutes to hours. Even 10,000 requests per hour cost only seconds of CPU.
It removes the actual problem	The cluster currently stops until an administrator intervenes; avoidance guarantees a safe sequence always exists, so no job is aborted and no work is lost, unlike detection (12 job-minutes lost in the demo).
Better than all-at-once allocation	Resources are handed out incrementally when safe, so GPUs do not sit idle waiting for the last licence.

Limitations and why the safety net is still needed: the Banker's algorithm assumes a fixed number of resources and jobs and accurate maximum claims, and it is conservative (it may delay requests that would have been safe). For resources outside the scheduler's control (for example a lock inside an application), deadlocks can still happen, so a detection run every few minutes (cost ≈ 913 µs at n=1,000) with abort of the least-work victim and automatic restart from checkpoints provides recovery. If maximum demands could not be declared, I would fall back to prevention through resource ordering plus detection and recovery.

Limitations of this report

The allocation matrices are the standard textbook example because the assignment file did not include them. Timings come from a worst-case constructed state on a 2-core virtualised container, with about ±20% run-to-run variation. The detection demo is a small example, not a full cluster simulation. Run the programs yourself, commit `Task06/banker.c, detect.c` with outputs, and add your name, register number and GitHub link.